

BAN404 R Exa Help

Comprehensive solution guide based on dataset and possible questions

Self-study preparation file

2026-01-01

Table of contents

1	How to use this file in the exam	6
1.1	The exam workflow from A to Z	6
2	BAN404 method recognition table	7
3	Universal data loading cookbook	7
3.1	Safe package setup	7
3.2	Safe CSV loading templates	8
3.2.1	Case 1: normal comma-separated file	8
3.2.2	Case 2: semicolon-separated file	8
3.2.3	Case 3: first real row does not start at A1	8
3.2.4	Case 4: weird missing values	8
3.3	Safe cleaning template	8
3.4	Common exam data problems → safest fixes	9
3.5	Typical exam wrangling questions → what they are really asking	9
3.6	Dataset-type clues that should shape your first thoughts	10
3.7	How to read the help page for common BAN404 functions	10
4	Universal wrangling and split templates	11
4.1	Live 2026 setup objects used in the dataset-tailored templates below	11
4.2	Create or modify columns safely	12
4.3	Recode factors safely	12
4.4	Standard 50/50 split	13
4.5	Standard 70/30 split	13
4.6	Model matrix for <code>glmnet</code>	13
4.7	When to use full data, training data, test data, or CV	14
5	Metric map	14
5.1	Regression metrics	14
5.2	Classification metrics	15
5.3	Unsupervised metrics	15
5.4	If the histogram, QQ-plot, or scatterplot does not look nice	15
5.5	BAN404-safe model building ladder	16
5.5.1	If the response is continuous	16
5.5.2	If the response is binary	16
6	High-probability method playbooks	16
6.1	1. OLS / linear regression	16
6.1.1	What exam wording signals it?	16
6.1.2	When to use it	17
6.1.3	Response type	17
6.1.4	Core theory in easy words	17
6.1.5	Assumptions and weaknesses	17
6.1.6	What to tune	17
6.1.7	How to evaluate	17

6.1.8	How to improve it	17
6.1.9	How to interpret it	17
6.1.10	Full runnable template	18
6.1.11	Typical BAN404 variations	18
6.1.12	Common traps	18
6.1.13	Past exam links	19
6.2	2. Logistic regression	19
6.2.1	What exam wording signals it?	19
6.2.2	When to use it	19
6.2.3	Response type	19
6.2.4	Core theory in easy words	19
6.2.5	Assumptions and weaknesses	19
6.2.6	What to tune	19
6.2.7	What to check before trusting the model	20
6.2.8	Base logistic $\rightarrow$ improved logistic $\rightarrow$ flexible classifier	20
6.2.9	How to evaluate	20
6.2.10	Full runnable template	20
6.2.11	Threshold tuning template for rare-event classification	21
6.2.12	How to improve it	22
6.2.13	How to interpret it	22
6.2.14	Typical BAN404 variations	22
6.2.15	Common traps	22
6.2.16	Past exam links	22
6.3	3. LOOCV and k-fold CV	22
6.3.1	What exam wording signals it?	22
6.3.2	When to use it	23
6.3.3	Response type	23
6.3.4	Core theory in easy words	23
6.3.5	Assumptions and weaknesses	23
6.3.6	What to tune	23
6.3.7	How to evaluate	23
6.3.8	How to improve it	23
6.3.9	How to interpret it	23
6.3.10	Generic LOOCV skeleton	23
6.3.11	Generic tuning loop	24
6.3.12	Past exam links	24
6.4	4. Bootstrap	24
6.4.1	What exam wording signals it?	24
6.4.2	When to use it	24
6.4.3	Response type	24
6.4.4	Core theory in easy words	25
6.4.5	Assumptions and weaknesses	25
6.4.6	What to tune	25
6.4.7	How to evaluate	25
6.4.8	How to improve it	25
6.4.9	How to interpret it	25
6.4.10	Bootstrap a variance or mean	25
6.4.11	Bootstrap a regression coefficient	26
6.4.12	Past exam links	26
6.5	5. Ridge regression	27
6.5.1	What exam wording signals it?	27
6.5.2	When to use it	27
6.5.3	Response type	27
6.5.4	Core theory in easy words	27
6.5.5	Assumptions and weaknesses	27
6.5.6	What to tune	27
6.5.7	How to evaluate	27
6.5.8	BAN404-style custom ridge template	27
6.5.9	Common BAN404 written point	28
6.5.10	How to improve it	28
6.5.11	How to interpret it	28

	6.5.12	Past exam links	28
6.6	6.	LASSO	28
	6.6.1	What exam wording signals it?	28
	6.6.2	When to use it	28
	6.6.3	Response type	28
	6.6.4	Core theory in easy words	29
	6.6.5	Assumptions and weaknesses	29
	6.6.6	What to tune	29
	6.6.7	How to evaluate	29
	6.6.8	Full runnable template	29
	6.6.9	How to improve it	30
	6.6.10	How to interpret it	30
	6.6.11	Past exam links	30
6.7	7.	GAM / smoothing splines / backfitting	30
	6.7.1	What exam wording signals it?	30
	6.7.2	When to use it	30
	6.7.3	Response type	30
	6.7.4	Core theory in easy words	30
	6.7.5	Assumptions and weaknesses	30
	6.7.6	What to tune	30
	6.7.7	How to evaluate	31
	6.7.8	Full runnable template	31
	6.7.9	Backfitting in plain English	33
	6.7.10	How to improve it	34
	6.7.11	How to interpret it	34
	6.7.12	Past exam links	34
6.8	8.	KNN / local regression	34
	6.8.1	What exam wording signals it?	34
	6.8.2	When to use it	34
	6.8.3	Response type	34
	6.8.4	Core theory in easy words	34
	6.8.5	Assumptions and weaknesses	34
	6.8.6	What to tune	34
	6.8.7	One-predictor local regression template	35
	6.8.8	Tune K with LOOCV	35
	6.8.9	Multi-predictor note	35
	6.8.10	How to evaluate	35
	6.8.11	How to improve it	35
	6.8.12	How to interpret it	35
	6.8.13	Past exam links	36
6.9	9.	Regression tree	36
	6.9.1	What exam wording signals it?	36
	6.9.2	When to use it	36
	6.9.3	Response type	36
	6.9.4	Core theory in easy words	36
	6.9.5	Assumptions and weaknesses	36
	6.9.6	What to tune	36
	6.9.7	Full runnable template	36
	6.9.8	How to evaluate	37
	6.9.9	How to improve it	37
	6.9.10	How to interpret it	37
	6.9.11	Past exam links	37
6.10	10.	Classification tree	37
	6.10.1	What exam wording signals it?	37
	6.10.2	When to use it	37
	6.10.3	Response type	38
	6.10.4	Core theory in easy words	38
	6.10.5	Assumptions and weaknesses	38
	6.10.6	What to tune	38
	6.10.7	Full runnable template	38
	6.10.8	How to evaluate	39

6.10.9	How to improve it	39
6.10.10	How to interpret it	39
6.10.11	Past exam links	39
6.11	11. Bagging	39
6.11.1	What exam wording signals it?	39
6.11.2	When to use it	39
6.11.3	Response type	39
6.11.4	Core theory in easy words	39
6.11.5	Assumptions and weaknesses	40
6.11.6	What to tune	40
6.11.7	Safe bagging approximation template	40
6.11.8	How to evaluate	40
6.11.9	How to improve it	40
6.11.10	How to interpret it	40
6.11.11	Past exam links	40
6.12	12. Random forest	41
6.12.1	What exam wording signals it?	41
6.12.2	When to use it	41
6.12.3	Response type	41
6.12.4	Core theory in easy words	41
6.12.5	Assumptions and weaknesses	41
6.12.6	What to tune	41
6.12.7	Classification template	41
6.12.8	Regression template	42
6.12.9	How to evaluate	42
6.12.10	How to improve it	42
6.12.11	How to interpret it	42
6.12.12	Past exam links	42
6.13	13. Boosting	43
6.13.1	What exam wording signals it?	43
6.13.2	When to use it	43
6.13.3	Response type	43
6.13.4	Core theory in easy words	43
6.13.5	Assumptions and weaknesses	43
6.13.6	What to tune	43
6.13.7	Classification template	43
6.13.8	Regression template	44
6.13.9	How to evaluate	44
6.13.10	How to improve it	44
6.13.11	How to interpret it	45
6.13.12	Past exam links	45
7	Lower-probability fallback sections	45
7.1	LDA / QDA	45
7.2	SVM	45
7.3	PCA / PCR	46
7.4	K-means	46
8	Question wording → action map	47
8.1	15 code lines that rescue you in BAN404	47
8.2	Base model → improvement model → interpretation guide	47
9	Written answer templates	48
9.1	Why ridge differs from OLS	48
9.2	Why demeaning is needed in BAN404-style ridge	48
9.3	Why train/test is used	48
9.4	Why CV is used	48
9.5	Why a variable is removed	48
9.6	Why GAM helps	48
9.7	How to interpret variable importance in random forest	49

10 2026 exam data reconnaissance — first pivot to the live datasets	49
10.1 Fast map of the two live datasets	49
10.2 Dataset 1 — <code>task1_data.csv</code>	49
10.2.1 What we already know before fitting anything advanced	49
10.2.2 Why this matters for likely Task 1 wording	50
10.2.3 First-pass working code for Task 1	50
10.2.4 Short written answer you can reuse for Task 1	52
10.3 Dataset 2 — <code>customer_data.csv</code>	53
10.3.1 What the dataset is already telling us	53
10.3.2 Why this matters for likely Task 2 wording	53
10.3.3 First-pass working code for Task 2	53
10.3.4 Short written answer you can reuse for Task 2	57
10.4 Three very likely live-data question scenarios	57
10.4.1 Scenario 1 — Task 1 asks for a safe baseline plus one improvement model	57
10.4.2 Scenario 2 — Task 2 asks which variables are associated with churn	57
10.4.3 Scenario 3 — Task 2 asks for the best predictive model	58
10.5 RStudio snippets to save for plug-and-play tomorrow	58
11 Dataset-anchored worked examples for likely exam questions	58
11.1 Fast map: which local dataset is useful for which model family?	58
11.2 Example 1 — 2025 Task 1 style: <code>data_task1.csv</code>	59
11.2.1 Why this is a high-priority example	59
11.2.2 What kind of exam wording this example prepares you for	59
11.2.3 Full worked code skeleton	59
11.2.4 Why these code chunks are the right choices	62
11.2.5 Ready-to-use written explanation	63
11.3 Example 2 — 2024 Task 2 style: <code>airline.csv</code>	63
11.3.1 Why this is a high-priority example	63
11.3.2 Full worked code skeleton	63
11.3.3 Why these code chunks are the right choices	66
11.3.4 Ready-to-use written explanation	66
11.4 Example 3 — 2023 style: <code>Churn.csv</code> teaches response-type thinking	66
11.4.1 Why this example matters	66
11.4.2 Full worked code skeleton	66
11.4.3 Why these code chunks are the right choices	68
11.4.4 Ready-to-use written explanation	68
11.5 Example 4 — 2025 Task 2 style: <code>insuranceData::dataOhlsson</code>	68
11.5.1 Why this is one of the most important examples	68
11.5.2 How the exam might ask this kind of question	69
11.5.3 What you should immediately think when you see a dataset like this	69
11.5.4 Full worked code skeleton	69
11.5.5 How to think about likely exam variants	73
11.5.6 How this also helps with loan-default datasets	74
11.5.7 Ready-to-use written explanation	74
11.6 Ready-to-use reasoning blocks for model and resampling choice	74
11.6.1 Why logistic regression instead of OLS?	74
11.6.2 Why LOOCV instead of a simple train/test split?	74
11.6.3 Why bootstrap instead of cross-validation?	74
11.6.4 Why random forest after logistic regression or a single tree?	75
11.6.5 Why remove a variable for the operational prediction version?	75
12 Mini exam-day checklist	75
12.1 Before coding	75
12.2 Before modeling	75
12.3 Before final answer	75
13 Appendix A — Maximum-output ChatGPT prompt	75
14 Appendix B — Maximum-output Claude prompt	75

1 How to use this file in the exam

This file is meant to be the **one big safety net** for BAN404. It is written for a student who may understand the course ideas better than the R syntax, package names, or full modeling workflow.

1.1 The exam workflow from A to Z

1. Find the response type first

- If y is numeric/continuous, think **regression**.
- If y is yes/no, 0/1, churn/not churn, satisfied/dissatisfied, think **classification**.
- If there is no y , think **unsupervised learning** like PCA or k-means.

2. Find the goal of the question

- “Explain what the code does” → identify the method from the code tell.
- “Predict” → build model + evaluate on test data.
- “Which variables matter?” → use EDA + interpretable model.
- “Find optimal K / λ / threshold” → use CV or LOOCV.

3. Pick the correct evaluation metric

- Continuous y → usually **MSE**, sometimes **R-squared**.
- Binary/categorical y → usually **confusion matrix** + **accuracy**, sometimes threshold discussion.
- Unsupervised → **variance explained**, **scree plot**, or **elbow plot**.

4. Decide which data to use

- Explanatory question → usually training data or all data if no prediction claim is being made.
- Predictive question → hold out test data.
- Tuning question → CV/LOOCV on training data.
- Final score comparison → same split and same metric for all models.

5. Choose the safest BAN404 answer

- Start with the course baseline.
- Then compare with the improved method the question is hinting at.
- Do not overcomplicate if a simple valid model answers the exact question.

6. Always finish with plain-language interpretation

- What did the model say?
 - Did it improve prediction?
 - Why was the metric appropriate?
 - Which variables or patterns mattered most?
-

2 BAN404 method recognition table

If the question or code shows...	It is probably...	Tune what?	Evaluate with...	Seen in BAN404
<code>sum((y - X%*%b)^2) + la*sum(b^2)</code>	Ridge regression	<code>lambda</code>	Test/LOOCV MSE	2025
<code>sum(abs(b))</code> or <code>alpha = 1</code> in <code>glmnet</code>	LASSO	<code>lambda</code>	Test/CV MSE	2023
<code>X[-i,]</code> or <code>for(i in 1:n)</code>	LOOCV	<code>lambda</code> , <code>K</code> , threshold	CV MSE / CV accuracy	2024, 2025
<code>sample(..., replace = TRUE)</code>	Bootstrap or bagging	<code>B</code> sometimes	CI / SE / accuracy / MSE	2021, 2023, 2025
<code>order(d)[1:K]</code>	KNN / local regression	<code>K</code>	LOOCV MSE / accuracy	2024
<code>s(x)</code> or <code>smooth.spline(...)</code>	GAM / smoothing spline	degrees of freedom / smoothness	MSE / R-squared	2021, 2022, 2025
<code>glm(..., family = binomial)</code>	Logistic regression	threshold	confusion matrix / accuracy	2021–2025
<code>tree(...)</code>	Regression/classification tree	tree size / pruning	MSE or accuracy	2022, 2023, 2024
<code>randomForest(...)</code>	Random forest / bagging approximation	<code>mtry</code> , <code>ntree</code>	MSE / accuracy / variable importance	2022–2025
<code>gbm(...)</code>	Boosting	trees, depth, shrinkage	MSE / accuracy	2021, 2025
<code>prcomp(...)</code>	PCA	number of PCs	variance explained	theory / possible fallback
<code>svm(...)</code> or <code>cost</code> , <code>gamma</code>	SVM	<code>cost</code> , <code>gamma</code>	accuracy	theory / possible fallback
<code>kmeans(...)</code>	K-means	number of clusters <code>K</code>	WCSS / elbow	theory / possible fallback

3 Universal data loading cookbook

3.1 Safe package setup

```
# Load the tidyverse first because BAN404 answers should stay in one tidyverse-friendly workflow for data
library(tidyverse) # Core data tools used throughout the cheatsheet and many BAN404 workflows.

# Load modeling packages that BAN404 repeatedly uses in old exams. # We load them early so we do not load
library(glmnet)      # Needed for ridge and lasso with matrix input, as in 2023-style regularization tasks
library(gam)         # Needed for GAM and smoothing terms, especially 2025 Task 1e style questions.
library(tree)        # Needed for regression trees and classification trees from 2022–2024 style tasks
library(randomForest) # Needed for random forest and bagging-style tree ensembles in many Task 2 questions
library(gbm)         # Needed for boosted trees, which appeared clearly in 2021 and 2025.
library(MASS)        # Useful fallback package for LDA/QDA if theory or code asks for it.
library(e1071)       # Useful fallback package for SVM if BAN404 theory or a rare exam twist asks for it.
library(insuranceData) # Needed for the 2025 insurance task because `dataOhlsson` lives in this package
```

3.2 Safe CSV loading templates

3.2.1 Case 1: normal comma-separated file

```
# Read a standard comma-separated csv. # This is the safest starting point when the exam file looks normal.
df <- read_csv("my_data.csv", show_col_types = FALSE) # read_csv guesses column types and is tidyverse
```

Error: 'my_data.csv' does not exist in current working directory ('C:/Users/willi/OneDrive/Documents/NH')

```
# Inspect structure immediately. # We do this to see whether columns became character, numeric, or logical.
glimpse(df) # glimpse is fast and exam-friendly for checking whether import went wrong.
```

```
function (x, df1, df2, ncp, log = FALSE)
```

3.2.2 Case 2: semicolon-separated file

```
# Read a semicolon-separated csv. # This is important because the 2025 exam explicitly used read.csv2
df <- read_csv2("my_data.csv", show_col_types = FALSE) # read_csv2 expects semicolons and comma decimal
```

Error: 'my_data.csv' does not exist in current working directory ('C:/Users/willi/OneDrive/Documents/NH')

```
# Check the import right away. # If everything became one big text column, the separator assumption was wrong.
glimpse(df) # Always verify before modeling.
```

```
function (x, df1, df2, ncp, log = FALSE)
```

3.2.3 Case 3: first real row does not start at A1

```
# Skip junk rows above the real header. # This solves the exam twist where the actual table starts below row 3.
df <- read_csv("my_data.csv", skip = 3, show_col_types = FALSE) # skip = 3 means ignore the first three rows
```

Error: 'my_data.csv' does not exist in current working directory ('C:/Users/willi/OneDrive/Documents/NH')

```
# Re-check that column names now make sense. # If names are still wrong, try a different skip value.
colnames(df) # Column names are the first clue that the import worked.
```

NULL

3.2.4 Case 4: weird missing values

```
# Tell read_csv which strings should count as NA. # This prevents text placeholders from breaking summaries.
df <- read_csv(
  "my_data.csv", # File path to the exam dataset.
  na = c("", "NA", "N/A", "Missing", "?"), # Common non-standard missing value strings.
  show_col_types = FALSE # Keeps the output cleaner during the exam.
)
```

Error: 'my_data.csv' does not exist in current working directory ('C:/Users/willi/OneDrive/Documents/NH')

3.3 Safe cleaning template


```
# Start with a raw dataset object. # Keeping a raw copy lets us restart if we make a cleaning mistake.
df_raw <- read_csv("my_data.csv", show_col_types = FALSE) # Always keep the untouched original.
```

Error: 'my_data.csv' does not exist in current working directory ('C:/Users/willi/OneDrive/Documents/NH

```
# Create a working copy for analysis. # This is where all changes should happen.
df <- df_raw %>%
  janitor::clean_names() %>% # Makes column names easier to type and les
  mutate(across(where(is.character), as.factor)) # Converts text columns to factors when the
```

Error: object 'df_raw' not found

```
# Check what happened. # We need to verify whether any variable was converted in a way we do not want.
glimpse(df)
```

```
function (x, df1, df2, ncp, log = FALSE)
```

3.4 Common exam data problems → safest fixes

Problem	Safe fix
Header not on first row	use <code>skip = ...</code> in <code>read_csv()</code> or <code>read_csv2()</code>
Semicolon file	use <code>read_csv2()</code>
Decimal comma	use <code>read_csv2()</code>
Text placeholders for missing values	use <code>na = c(...)</code>
ID column like <code>id</code> or <code>customer_id</code>	remove with <code>select(-id)</code>
Binary target stored as yes/no text	convert to factor with clear levels
Binary target stored as 0/1	keep numeric for some models or convert to factor for classification summaries
Character predictors	convert to factor with <code>mutate(across(where(is.character), as.factor))</code>
Leakage variable	remove before train/test split and explain why
Future-unavailable variable	remove if prediction is for new customers/future observations
<code>glmnet</code> with factors	create dummy variables using <code>model.matrix()</code>
Distance-based method	standardize predictors before KNN/PCA/SVM/k-means

3.5 Typical exam wrangling questions → what they are really asking

If the exam says...	What they really want you to do
“Recode variables as needed”	find characters/binaries/ordinal codes that should become factors
“Remove unreasonable variables”	remove IDs, leakage variables, and unrealistic future-only variables
“Use descriptive statistics to find useful predictors”	compare outcome groups with tables, proportions, boxplots, and grouped summaries
“Motivate your preprocessing”	explain every removal or recode in business terms, not only in R terms
“Create a binary target”	turn counts or text labels into a factor with explicit levels like <code>c("no", "yes")</code>
“Use the dataset documentation”	inspect <code>help(dataset_name)</code> and translate variable meanings into modeling decisions

3.6 Dataset-type clues that should shape your first thoughts

If the dataset looks like...	Likely response type	Early warning sign	Usual first thought
churn / claim / default / fraud	binary classification	class imbalance + threshold choice	logistic baseline, confusion matrix, sensitivity/specificity
satisfaction / yes-no survey outcome	binary classification	leakage from post-experience variables	separate explanatory vs operational questions
price / bill / cost / score	continuous regression	skewness, nonlinearity, outliers	OLS baseline, then ridge/lasso/GAM/tree if needed
count of events	count or engineered binary	many zeros	consider whether BAN404 wants a binary engineered target
many service ratings + final outcome	mixed predictors	redundant highly related predictors	baseline interpretable model, then flexible improvement model

If you get a **loan default** or **loan approval** dataset, think very similarly to churn or insurance claims:

- the response is often **binary** (default / no default, approved / rejected),
- the positive class may be **rare**, so plain accuracy can mislead,
- variables measured **after** the decision or after the loan period are likely leakage,
- threshold choice matters because false negatives and false positives may have different business costs.

3.7 How to read the help page for common BAN404 functions

In an exam, the help file often tells you **what the first inputs mean**, which is enough to rescue you even if you forget the full syntax.

Function	First important inputs	What they mean in plain English
<code>read_csv(file, ...)</code>	<code>file</code>	the path to the csv you want to import
<code>read_csv2(file, ...)</code>	<code>file</code>	same idea, but meant for semicolon-style csv files
<code>mutate(.data, ...)</code> <code>select(.data, ...)</code>	<code>.data</code> <code>.data</code>	the data frame you are modifying the data frame you are keeping/removing columns from
<code>lm(formula, data)</code>	<code>formula, data</code>	what to predict from what, and which data frame to use
<code>glm(formula, data, family)</code>	<code>formula, data, family</code>	same as <code>lm</code> , plus which response-model family to use
<code>predict(object, newdata, type)</code>	<code>object, newdata, type</code>	fitted model, data to predict on, and prediction scale
<code>cv.glmnet(x, y, ...)</code>	<code>x, y</code>	predictor matrix and response vector
<code>randomForest(formula, data, ...)</code>	<code>formula, data</code>	model formula and training data
<code>gbm(formula, data, distribution, ...)</code>	<code>formula, data, distribution</code>	what to predict, from what data, and whether the task is regression or classification
<code>sample(x, size, replace)</code>	<code>x, size, replace</code>	what to sample from, how many draws, and whether resampling is allowed

```
# Open the documentation page for a dataset or function. # This is often enough to recover the variable
help(dataOhlsson) # Shows what the insurance variables mean, which helps with leakage and modeling choices

# Ask R to print the argument names for a function. # This is a fast way to remember the important inputs
args(glm) # Shows that glm mainly wants a formula, a data frame, and a family.
```

```
function (formula, family = gaussian, data, weights, subset,
  na.action, start = NULL, etastart, mustart, offset, control = list(...),
  model = TRUE, method = "glm.fit", x = FALSE, y = TRUE, singular.ok = TRUE,
  contrasts = NULL, ...)
NULL
```

```
args(predict) # Shows that predict mainly wants a fitted object and newdata.
```

```
function (object, ...)
NULL
```

```
args(randomForest) # Helpful when you forget where mtry or ntree belongs.
```

```
function (x, ...)
NULL
```

4 Universal wrangling and split templates

4.1 Live 2026 setup objects used in the dataset-tailored templates below

```
# Read the live Task 1 dataset that is actually attached for the 2026 exam. # We keep the path short because
task1_2026 <- read_csv("task1_data.csv", show_col_types = FALSE) # Continuous-response dataset for likelihood

# Read the live Task 2 dataset that is actually attached for the 2026 exam. # This is the churn/customer
customer_2026_raw <- read_csv("customer_data.csv", show_col_types = FALSE) # Raw customer data before cleaning

# Clean and recode the customer dataset into a model-ready BAN404 classification table. # We keep the
customer_2026 <- customer_2026_raw %>%
  janitor::clean_names() %>% # Convert column names to lower snake_case so later code is easier to type
  mutate(
    churn = factor(churn, levels = c(FALSE, TRUE), labels = c("no", "yes")), # Make the binary target
    international_plan = factor(international_plan), # Yes/no plan indicator should be categorical.
    voice_mail_plan = factor(voice_mail_plan), # Same logic for voice-mail subscription.
    state = factor(state), # State is a category, not a numeric scale.
    area_code = factor(area_code) # Area code is better handled as a category than a number
  ) %>%
  select(
    -total_day_charge, # Drop redundant charge column because total_day_minutes already contains the
    -total_eve_charge, # Same redundancy logic for evening usage.
    -total_night_charge, # Same redundancy logic for night usage.
    -total_intl_charge # Same redundancy logic for international usage.
  )
```

Error in select(., -total_day_charge, -total_eve_charge, -total_night_charge, : unused arguments (-total_eve_charge, -total_night_charge, -total_intl_charge)

```
# Create a reproducible Task 1 split. # We use one consistent split so all Task 1 model comparisons are
set.seed(404) # Fixed seed so your written answer can be reproduced later.
idx_task1_2026 <- sample(seq_len(nrow(task1_2026)), size = floor(0.70 * nrow(task1_2026))) # 70 percent
task1_train_2026 <- task1_2026[idx_task1_2026, ] # Training data for Task 1 models.
task1_test_2026 <- task1_2026[-idx_task1_2026, ] # Test data for honest out-of-sample evaluation.
```

```
# Create a reproducible Task 2 split. # We use the same split across logistic, tree, RF, and boosting
idx_customer_2026 <- sample(seq_len(nrow(customer_2026)), size = floor(0.70 * nrow(customer_2026))) #
```

Error: object 'customer_2026' not found

```
customer_train_2026 <- customer_2026[idx_customer_2026, ] # Training customers.
```

Error: object 'customer_2026' not found

```
customer_test_2026 <- customer_2026[-idx_customer_2026, ] # Test customers.
```

Error: object 'customer_2026' not found

```
# Build full-data Task 1 matrix objects for custom ridge/L00CV style questions. # Some BAN404 Task 1 q
X_task1_full_2026 <- task1_2026 %>% select(-y) %>% as.matrix() # Predictor matrix for formula-free cus
```

Error in h(simpleError(msg, call)): error in evaluating the argument 'x' in selecting a method for func

```
y_task1_full_2026 <- task1_2026$y # Continuous response vector.
```

```
# Build train/test Task 1 matrices for glmnet-based ridge and LASSO. # glmnet expects numeric matrices
X_task1_train_2026 <- model.matrix(y ~ ., data = task1_train_2026)[, -1] # Dummy-ready numeric training
y_task1_train_2026 <- task1_train_2026$y # Training response vector.
X_task1_test_2026 <- model.matrix(y ~ ., data = task1_test_2026)[, -1] # Matching numeric test matr
y_task1_test_2026 <- task1_test_2026$y # Test response vector.
```

4.2 Create or modify columns safely

```
# Create a binary claim variable. # This exact type of target engineering appeared in the 2025 insuran
df <- df %>%
  mutate(claim = if_else(antskad >= 1, "yes", "no")) %>% # Translate count outcome into binary event i
  mutate(claim = factor(claim, levels = c("no", "yes"))) # Make the reference class explicit for later
```

Error in UseMethod("mutate"): no applicable method for 'mutate' applied to an object of class "function"

```
# Remove obvious leakage or useless variables. # BAN404 repeatedly tests whether you can identify unre
df <- df %>%
  select(-id, -antskad, -skadkost) # Remove identifiers and post-outcome variables that would leak tar
```

Error in select(., -id, -antskad, -skadkost): unused arguments (-id, -antskad, -skadkost)

4.3 Recode factors safely

```
# Convert selected variables to factors. # This is common in airline, churn, insurance, and OJ-style d
df <- df %>%
  mutate(
    gender = as.factor(gender),          # Gender is categorical, not numeric distance infor
    customer_type = as.factor(customer_type), # Customer type should be treated as categories.
    satisfaction = as.factor(satisfaction) # Binary or multiclass target should be explicit as
  )
```

Error in UseMethod("mutate"): no applicable method for 'mutate' applied to an object of class "function"

4.4 Standard 50/50 split

```
# Set a seed first. # Fixed seeds are important because BAN404 often gives one directly in the questio
set.seed(86554354) # Reproducibility matters for matching your written answer to your output.

# Create row indices for the training set. # We sample observation numbers, not the data itself.
idx <- sample(seq_len(nrow(df)), size = floor(nrow(df) / 2)) # Half the observations go to training.
```

Error in seq_len(nrow(df)): argument must be coercible to non-negative integer

```
# Build train and test sets. # This is the standard predictive workflow in many BAN404 tasks.
train <- df[idx, ] # Training data is used for fitting and CV.
```

Error: object 'idx' not found

```
test <- df[-idx, ] # Test data is held back for honest final evaluation.
```

Error: object 'idx' not found

4.5 Standard 70/30 split

```
# Use a 70/30 split when the question does not dictate 50/50 and you want more training signal. # This
set.seed(123) # Reproducible split.
idx <- sample(seq_len(nrow(df)), size = floor(0.70 * nrow(df))) # 70 percent train.
```

Error in seq_len(nrow(df)): argument must be coercible to non-negative integer

```
train <- df[idx, ] # Fit models here.
```

Error: object 'idx' not found

```
test <- df[-idx, ] # Evaluate final predictions here.
```

Error: object 'idx' not found

4.6 Model matrix for glmnet

```
# Build X and y for regularized regression on the live Task 1 data. # glmnet requires a numeric matrix
X_train_task1_2026 <- model.matrix(y ~ ., data = task1_train_2026)[, -1] # model.matrix creates dummy
y_train_task1_2026 <- task1_train_2026$y # Response stays as a numeric

# Build a test matrix using the same formula. # We need the same columns at prediction time.
X_test_task1_2026 <- model.matrix(y ~ ., data = task1_test_2026)[, -1] # Same dummy structure for te
y_test_task1_2026 <- task1_test_2026$y # True test response for evaluation
```

4.7 When to use full data, training data, test data, or CV

Situation	Safest BAN404 choice
“Why are customers dissatisfied?”	training data or all data for explanation
“Predict new customers”	train/test split
“Find optimal lambda/K/threshold”	CV or LOOCV on training data
“Compare models fairly”	same training/test split and same metric
Very small dataset	LOOCV or k-fold CV
Final performance claim	test data

5 Metric map

5.1 Regression metrics

Metric	Use when	What it means	Typical BAN404 use
MSE	continuous y	average squared prediction error	OLS, ridge, lasso, GAM, trees
RMSE	continuous y	square root of MSE, same units as y	optional readable supplement
R-squared	continuous y	share of variation explained	2025 Task 1e
Adjusted R-squared	OLS model fit comparison	penalizes too many predictors	optional explanatory answer

```
# Compute test MSE. # This is the most common BAN404 regression metric.
mse_test <- mean((y_test - y_pred)^2) # Squaring punishes large errors more than small ones.
```

Error in h(simpleError(msg, call)): error in evaluating the argument 'x' in selecting a method for function h

```
# Compute RMSE. # Useful when you want the error back in the original units of y.
rmse_test <- sqrt(mse_test) # Easier to interpret than MSE, though less emphasized in BAN404.
```

Error: object 'mse_test' not found

```
# Compute R-squared from fitted values. # This is useful when the exam directly asks for explained variance
r2 <- 1 - sum((y_test - y_pred)^2) / sum((y_test - mean(y_test))^2) # Standard out-of-sample R-squared
```

Error: object 'y_test' not found

5.2 Classification metrics

Metric	Use when	What it means	Typical BAN404 use
Confusion matrix	classification	table of true vs predicted classes	every Task 2 classification pipeline
Accuracy	classification	share classified correctly	2021–2025
Error rate	classification	1 - accuracy	same story, just reversed
Sensitivity	costly false negatives	share of positives correctly found	threshold discussion
Specificity	costly false positives	share of negatives correctly found	threshold discussion

```
# Build a confusion matrix. # This is the standard first classification evaluation step in BAN404.
cm <- table(actual = y_test, predicted = y_pred_class) # Rows are truth, columns are predictions.
```

Error: object 'y_test' not found

```
# Compute accuracy. # Accuracy is the most common summary metric in the old exams.
accuracy <- sum(diag(cm)) / sum(cm) # Correct predictions divided by all predictions.
```

Error in as.integer(x): cannot coerce type 'closure' to vector of type 'integer'

```
# Compute error rate. # Sometimes useful if the theory talks about classification error.
error_rate <- 1 - accuracy # Lower is better.
```

Error: object 'accuracy' not found

5.3 Unsupervised metrics

Method	Main metric	What to show
PCA	variance explained	scree plot + cumulative explained variance
K-means	within-cluster sum of squares	elbow plot

5.4 If the histogram, QQ-plot, or scatterplot does not look nice

Do **not** panic and do **not** assume the whole method is impossible.

- If a **predictor histogram** is not normal, that alone does **not** kill OLS or logistic regression.
- For **logistic regression**, the predictors do **not** need to be normally distributed.
- For **OLS**, what matters more is whether the **mean relationship looks roughly linear** and whether the **residual behavior** is acceptable for the task.
- If the response or residuals are strongly skewed, you can often still:
 - keep OLS as a baseline,
 - try $\log(y)$ if that makes business sense,
 - use bootstrap for uncertainty instead of relying too heavily on normal approximations,
 - move to GAM, trees, random forest, or boosting if nonlinearity is the real issue.
- If a histogram shows a **huge pile at zero** and a **small positive class**, think claim/default/churn-style classification logic, not “force normality.”

```
# Inspect a variable with a histogram. # Histograms help us understand shape, skewness, and zero-infla
hist(df$x) # A weird shape is a clue about the data, not an automatic reason to stop.
```

```
Error in df$x: object of type 'closure' is not subsettable
```

```
# Inspect residuals for an OLS model. # Residual diagnostics matter more than predictor normality.  
hist(residuals(ols_fit)) # If this is skewed, OLS can still be a baseline, but you may discuss transfo
```

```
Error: object 'ols_fit' not found
```

```
# Compare a linear and nonlinear fit visually. # Curvature suggests GAM or tree-based models may help.  
df %>%  
  ggplot(aes(x = x, y = y)) +  
  geom_point(alpha = 0.4) +  
  geom_smooth(method = "lm", se = FALSE, color = "blue") +  
  geom_smooth(se = FALSE, color = "red") # Big gap between blue and red suggests the linear form may b
```

```
Error in `ggplot()`:  
! `data` cannot be a function.  
i Have you misspelled the `data` argument in `ggplot()`
```

5.5 BAN404-safe model building ladder

5.5.1 If the response is continuous

1. Start with **OLS** as the clean benchmark.
2. If predictors are many/correlated, compare with **ridge** or **LASSO**.
3. If scatterplots show curvature, compare with **GAM**.
4. If interactions/nonlinearity seem strong, compare with **tree**, **random forest**, or **boosting**.
5. Evaluate all candidates with the **same split and same regression metric**.

5.5.2 If the response is binary

1. Start with **logistic regression** as the interpretable baseline.
 2. Check **class balance** before trusting accuracy.
 3. Tune the **threshold** on training/CV data if positives are rare or costs are asymmetric.
 4. Compare with **classification tree**, **random forest**, and **boosting** if prediction can likely improve through nonlinearities.
 5. Evaluate with **confusion matrix**, **sensitivity**, **specificity**, and **accuracy** in a way that matches the exam question.
-

6 High-probability method playbooks

6.1 1. OLS / linear regression

6.1.1 What exam wording signals it?

- “Fit an OLS model”

- “Use `lm()`”
- “Predict a continuous variable”
- “Compare with ridge / lasso / GAM”

6.1.2 When to use it

Use OLS when the response is continuous and you want the cleanest baseline model. In BAN404, OLS is often the **starting benchmark** before a more flexible or regularized model.

6.1.3 Response type

Continuous numeric outcome.

6.1.4 Core theory in easy words

OLS draws the line or hyperplane that makes the residual sum of squares as small as possible.

6.1.5 Assumptions and weaknesses

- y should be **continuous**, not binary.
- The **average relationship** should be roughly linear unless you deliberately upgrade the model.
- Observations should be reasonably **independent**.
- Strong multicollinearity can make coefficients unstable.
- OLS does **not** require every predictor to look normal in a histogram.

6.1.6 What to tune

- Plain OLS has no main tuning parameter.
- The practical choices are usually **which predictors to include**, whether to **transform y** , and whether the model needs a **more flexible follow-up method**.

6.1.7 How to evaluate

- Use **MSE** or **RMSE** on test data for prediction.
- Use **R-squared** when the exam explicitly asks about explained variation.
- For explanatory questions, also inspect **coefficient signs**, **standard errors**, and whether the direction makes business sense.

6.1.8 How to improve it

- If coefficients look unstable because predictors overlap heavily → try **ridge** or **LASSO**.
- If plots suggest curvature → try **GAM**.
- If you suspect interactions or threshold effects → try **tree-based models**.
- If the residual distribution looks ugly but the task is mainly predictive → still keep OLS as a baseline, then compare with more flexible models using the same test metric.

6.1.9 How to interpret it

- Each coefficient is the expected change in y from a one-unit change in that predictor, holding the other predictors fixed.
- The intercept is the expected y when all predictors are zero, though it is often less interesting than the slopes.
- If OLS is only the baseline, the main interpretation job is often: **what is the direction of the main effects, and how strong is the benchmark fit before improvement models are tested?**

6.1.10 Full runnable template

```
# Fit a multiple linear regression model on the live 2026 Task 1 data. # This is the standard baseline
ols_task1_2026 <- lm(y ~ ., data = task1_train_2026) # The dot means "use all remaining predictors in

# View the coefficient table. # This helps with interpretation and with deciding whether some variable
summary(ols_task1_2026) # Gives coefficients, standard errors, t-values, p-values, and model fit summary

Call:
lm(formula = y ~ ., data = task1_train_2026)

Residuals:
    Min       1Q   Median       3Q      Max
-4.831 -1.367 -0.364  0.946 42.881

Coefficients:
            Estimate Std. Error t value Pr(>|t|)
(Intercept)  3.48749    0.03834   90.955 <2e-16 ***
x1           1.05370    0.03803   27.705 <2e-16 ***
x2           0.04987    0.03839    1.299  0.194
x3          -1.62837    0.03799  -42.867 <2e-16 ***
x4          -0.03284    0.03883   -0.846  0.398
x5           1.99671    0.03801   52.537 <2e-16 ***
x6           0.03673    0.03849    0.954  0.340
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 2.268 on 3493 degrees of freedom
Multiple R-squared:  0.6104,    Adjusted R-squared:  0.6097
F-statistic:  912 on 6 and 3493 DF,  p-value: < 2.2e-16

# Predict on the held-out test set. # Test data gives an honest view of generalization performance.
y_pred_task1_2026 <- predict(ols_task1_2026, newdata = task1_test_2026) # Use the fitted linear model

# Evaluate with test MSE. # Continuous responses should usually be scored with MSE in BAN404.
mse_ols_task1_2026 <- mean((task1_test_2026$y - y_pred_task1_2026)^2) # Smaller MSE means better prediction

# Optional: compute test R-squared. # Useful when the exam asks for explained variation.
r2_ols_task1_2026 <- 1 - sum((task1_test_2026$y - y_pred_task1_2026)^2) / sum((task1_test_2026$y - mean(task1_test_2026$y))^2)
```

6.1.11 Typical BAN404 variations

- If the response is skewed, try `lm(log(y) ~ ., data = train)`.
- If the question is explanatory, focus more on coefficients and less on squeezing out tiny predictive gains.
- If the question asks whether OLS is worse than ridge/lasso/GAM, compare **same split + same metric**.

6.1.12 Common traps

- Forgetting to remove leakage variables.
- Using accuracy or confusion matrix for a continuous y .
- Comparing training MSE for one model with test MSE for another.

6.1.13 Past exam links

- 2023 Task 1c
 - 2025 Task 1b baseline comparison
 - 2022 Task 2c and 2d
-

6.2 2. Logistic regression

6.2.1 What exam wording signals it?

- “Predict churn / claim / dissatisfaction”
- “Fit a logistic model”
- “Convert probabilities to 0/1 predictions”
- “Use confusion matrix and threshold”

6.2.2 When to use it

Use logistic regression when the response is binary and you want the safest interpretable baseline for classification.

6.2.3 Response type

Binary classification outcome, usually stored as 0/1 or as a two-level factor.

6.2.4 Core theory in easy words

Logistic regression predicts the **probability** that an observation belongs to class 1. Then you convert that probability into a class label using a threshold, often 0.5 unless the question suggests otherwise.

6.2.5 Assumptions and weaknesses

- The response should be **binary**.
- Observations should be reasonably **independent**.
- Numeric predictors are assumed to have a roughly **linear effect on the log-odds**, not necessarily on the raw probability.
- Very strong multicollinearity can make coefficients unstable.
- Perfect or near-perfect separation can break or distort the fit.
- Logistic regression does **not** require the predictors to be normally distributed.

6.2.6 What to tune

- The main practical tuning choice is the **classification threshold**.
- You may also tune the **predictor set** if the question asks for a reduced operational model.
- If plain logistic is too rigid, the next tuning step is often to compare with a more flexible classifier rather than over-complicating the logistic model itself.

6.2.7 What to check before trusting the model

```
# Check class balance first. # Rare positives often mean accuracy alone is misleading.  
prop.table(table(customer_train_2026$churn)) # This tells us whether the live customer churn problem is
```

Error: object 'customer_train_2026' not found

```
# Fit the logistic model on the live customer dataset. # We inspect the summary for coefficient direct  
logit_customer_2026 <- glm(churn ~ ., data = customer_train_2026, family = binomial()) # Standard bina
```

Error in eval(mf, parent.frame()): object 'customer_train_2026' not found

```
summary(logit_customer_2026) # Watch for huge standard errors, warnings, or suspicious coefficient mag
```

Error in h(simpleError(msg, call)): error in evaluating the argument 'object' in selecting a method for

```
# Look at predicted probabilities. # If nearly all are tiny, a 0.5 threshold may be inappropriate.  
pred_prob_customer_train_2026 <- predict(logit_customer_2026, newdata = customer_train_2026, type = "re
```

Error: object 'logit_customer_2026' not found

```
hist(pred_prob_customer_train_2026) # Rare-event data often produces many small fitted probabilities.
```

Error: object 'pred_prob_customer_train_2026' not found

6.2.8 Base logistic → improved logistic → flexible classifier

1. Fit a **baseline logistic regression**.
2. Check whether the data are **imbalanced**.
3. Tune the **threshold** if 0.5 is unrealistic.
4. If the exam asks for better prediction, compare with **classification tree**, **random forest**, or **boosting**.
5. If the exam asks “why”, lean on the logistic coefficients; if it asks “who should we flag”, lean more on threshold choice and predictive metrics.

6.2.9 How to evaluate

- Start with a **confusion matrix**.
- Then report **accuracy**, and when class imbalance matters also **sensitivity** and **specificity**.
- If the exam is about rare claims/defaults/churn, explain why threshold choice matters before claiming the model is good or bad.

6.2.10 Full runnable template

```
# Fit logistic regression for the live churn outcome. # family = binomial tells R this is a classifica  
logit_customer_2026 <- glm(churn ~ ., data = customer_train_2026, family = binomial()) # This is the s
```

Error in eval(mf, parent.frame()): object 'customer_train_2026' not found

```
# Inspect coefficients. # Signs tell direction and p-values help with explanatory discussion.
summary(logit_customer_2026) # Positive coefficient means higher log-odds of churn = yes, all else equ
```

Error in h(simpleError(msg, call)): error in evaluating the argument 'object' in selecting a method for

```
# Predict probabilities on test data. # Logistic regression naturally gives probabilities first, not c
pred_prob_customer_test_2026 <- predict(logit_customer_2026, newdata = customer_test_2026, type = "resp
```

Error: object 'logit_customer_2026' not found

```
# Choose a threshold. # 0.30 is a sensible live-data starting point because churn is the minority clas
threshold_customer_2026 <- 0.30 # A lower threshold catches more positives but may create more false p
```

```
# Convert probabilities to class predictions. # We now turn probability into the final yes/no answer.
pred_class_customer_2026 <- if_else(pred_prob_customer_test_2026 > threshold_customer_2026, "yes", "no"
  factor(levels = levels(customer_train_2026$churn)) # Matching levels avoids confusion matrix orderin
```

Error: object 'pred_prob_customer_test_2026' not found

```
# Evaluate predictions. # Confusion matrix plus accuracy is the standard BAN404 output.
cm_logit_customer_2026 <- table(actual = customer_test_2026$churn, predicted = pred_class_customer_2026
```

Error: object 'customer_test_2026' not found

```
acc_logit_customer_2026 <- sum(diag(cm_logit_customer_2026)) / sum(cm_logit_customer_2026)
```

Error in h(simpleError(msg, call)): error in evaluating the argument 'x' in selecting a method for func

6.2.11 Threshold tuning template for rare-event classification

```
# Create a small grid of candidate thresholds for the live churn data. # Rare-event data often needs a
threshold_grid_customer_2026 <- seq(0.10, 0.50, by = 0.05) # Search from clearly low to standard thres
```

```
# Score each threshold on training data. # Threshold choice should happen before touching final test p
threshold_scores_customer_2026 <- purrr::map_dfr(threshold_grid_customer_2026, function(th) {
  pred_class_train_customer_2026 <- if_else(pred_prob_customer_train_2026 > th, levels(customer_train_2
    factor(levels = levels(customer_train_2026$churn)) # Convert probabilities to classes using one ca
  cm_train_customer_2026 <- table(actual = customer_train_2026$churn, predicted = pred_class_train_cust
  tibble(
    threshold = th, # Candidate threshold value.
    sensitivity = cm_train_customer_2026[2, 2] / sum(cm_train_customer_2026[2, ]), # True positive rat
    specificity = cm_train_customer_2026[1, 1] / sum(cm_train_customer_2026[1, ]) # True negative rat
  )
})
```

Error in `map()`:
i In index: 1.
Caused by error in `.f()`:
! object 'pred_prob_customer_train_2026' not found

```
# Add balanced accuracy as a simple summary. # This is often more informative than plain accuracy in i
threshold_scores_customer_2026 <- threshold_scores_customer_2026 %>%
  mutate(balanced_accuracy = (sensitivity + specificity) / 2) # Average of the two class-wise hit rate
```

Error: object 'threshold_scores_customer_2026' not found

```
# Choose the best threshold by your chosen criterion. # Balanced accuracy is a safe exam-friendly default
best_threshold_customer_2026 <- threshold_scores_customer_2026 %>%
  slice_max(order_by = balanced_accuracy, n = 1) %>%
  pull(threshold) # Extract the winning threshold value.
```

Error: object 'threshold_scores_customer_2026' not found

6.2.12 How to improve it

- Remove **future-unavailable variables** for operational prediction.
- Tune the **threshold** instead of blindly using 0.5.
- Compare with **classification tree**, **random forest**, or **boosting** if nonlinearities or interactions may matter.
- If interpretation matters most, a reduced logistic model may be the best improvement rather than a black-box model.

6.2.13 How to interpret it

- A positive coefficient means the predictor increases the **log-odds** of the positive class, holding other predictors fixed.
- In plain language, you usually interpret the **direction** and importance rather than writing out log-odds algebra.
- The predicted probabilities matter operationally because they tell you **how strongly** each case looks like a positive outcome before thresholding.

6.2.14 Typical BAN404 variations

- **Explanatory logistic**: keep realistic predictors and discuss directions/significance.
- **Operational prediction logistic**: remove future-unavailable variables first.
- **Threshold discussion**: if false negatives are more costly, lower the threshold somewhat.

6.2.15 Common traps

- Forgetting `family = binomial()`.
- Using `predict()` without `type = "response"` and then treating log-odds as probabilities.
- Leaving leakage variables in when predicting future customers.

6.2.16 Past exam links

- 2021 Task 3
- 2022 Task 1c
- 2023 Task 2c–d
- 2024 Task 2d and 2j
- 2025 Task 2b–c

6.3 3. LOOCV and k-fold CV

6.3.1 What exam wording signals it?

- “Find optimal K”
- “Choose lambda”
- “Use cross-validation”
- “Explain what this loop does”

6.3.2 When to use it

Use CV when the exam is about **choosing a tuning parameter** or estimating generalization error without spending the full test set.

6.3.3 Response type

Not a model family by itself. It is a **resampling procedure** that can be used with both regression and classification.

6.3.4 Core theory in easy words

CV estimates how well a model generalizes to new data. LOOCV is the extreme version where you leave out one observation at a time.

6.3.5 Assumptions and weaknesses

- Observations should be reasonably **independent**.
- CV is for **tuning/evaluation**, not for estimating the sampling distribution of a statistic.
- LOOCV can be computationally slow if the underlying model fit is heavy.

6.3.6 What to tune

- Whatever the exam asks for: **K**, **lambda**, threshold, number of neighbors, tree size, or another model-setting choice.

6.3.7 How to evaluate

- Use **average CV error** for regression.
- Use **average CV accuracy** or another class metric for classification.
- The best tuning value is the one that optimizes the chosen CV score.

6.3.8 How to improve it

- Replace ad hoc guesses with a **grid search** over sensible candidate values.
- If the dataset is small, LOOCV is often safer than a simple single split.
- If LOOCV is too slow, a smaller **k-fold CV** version can be used.

6.3.9 How to interpret it

- The point is not the loop itself; the point is that CV gives an **approximately honest estimate** of how each tuning choice should generalize.
- In written answers, emphasize that CV helps choose the tuning value **before final test evaluation**.

6.3.10 Generic LOOCV skeleton

```
# Define a generic LOOCV function for a tuning parameter. # This is the classic BAN404 structure from
loo_model_task1_2026 <- function(tune_value, X, y) {      # tune_value could be lambda, K, threshold
  n <- nrow(X)                                           # Number of observations in the live Task
  pred <- numeric(n)                                    # Storage for one prediction per left-out

  for (i in seq_len(n)) {                               # Loop once per observation.
    X_train <- X[-i, , drop = FALSE]                   # Training predictors: all rows except i.
    y_train <- y[-i]                                    # Training response: all values except i.
```

```

X_test <- X[i, , drop = FALSE] # Test predictors: the left-out observation

# Replace the next line with the model fit that uses tune_value. # This is where ridge/KNN/etc. is
fit_i <- lm(y_train ~ ., data = as.data.frame(X_train)) # Placeholder example using OLS on the Tas

# Predict the held-out case. # LOOCV works by repeating this for every single row.
pred[i] <- predict(fit_i, newdata = as.data.frame(X_test))
}

mean((y - pred)^2) # Return LOOCV MSE so smaller is better.
}

```

6.3.11 Generic tuning loop

```

# Create candidate tuning values for the live Task 1 data. # The question usually wants the value that
param_grid_task1_2026 <- seq(0, 1000, length.out = 20) # Example grid for lambda or another continuous

# Score every candidate value. # sapply makes a compact loop over the grid.
cv_scores_task1_2026 <- sapply(param_grid_task1_2026, function(v) loo_model_task1_2026(v, X_task1_full_

```

Error in FUN(X[[i]], ...): object 'X_task1_full_2026' not found

```

# Pick the best tuning value. # Best means smallest error for regression or largest accuracy for class
best_value_task1_2026 <- param_grid_task1_2026[which.min(cv_scores_task1_2026)] # which.min finds the

```

Error: object 'cv_scores_task1_2026' not found

6.3.12 Past exam links

- 2024 Task 1b
- 2025 Task 1c
- 2022 threshold tuning
- 2021 Task 2c CV choice discussion

6.4 4. Bootstrap

6.4.1 What exam wording signals it?

- “Estimate the sampling distribution”
- “Compute a 95% confidence interval”
- “Bootstrap the standard error”
- “Show the result as a histogram”

6.4.2 When to use it

Use bootstrap when the target is **uncertainty about a statistic** such as a mean, variance, coefficient, or proportion.

6.4.3 Response type

Not a model family by itself. It is a **resampling method** used for inference-like uncertainty questions.

6.4.4 Core theory in easy words

The bootstrap pretends your sample is the population. It repeatedly samples **with replacement** and recalculates the statistic each time.

6.4.5 Assumptions and weaknesses

- The sample should be reasonably representative of the population you care about.
- Bootstrap is about the **sampling distribution of a statistic**, not about model tuning.
- If the data-generating structure is complex and the sample is tiny or highly dependent, bootstrap results can become unstable.

6.4.6 What to tune

- The practical tuning choice is usually **B**, the number of bootstrap resamples.
- Sometimes you also choose **which statistic** to store: a mean, variance, coefficient, proportion, and so on.

6.4.7 How to evaluate

- Look at the **histogram** of the bootstrap statistics.
- Report the **bootstrap standard error** and/or a **confidence interval**.
- If asked to compare methods, show both **percentile** and **normal-approximation** intervals.

6.4.8 How to improve it

- Increase **B** if the histogram or interval looks unstable.
- Resample **whole rows** when the statistic depends on relationships between predictors and response.
- Keep bootstrap separate from train/test prediction logic unless the question explicitly asks for a bootstrap prediction procedure.

6.4.9 How to interpret it

- The stored bootstrap values approximate how the statistic would vary across repeated samples from the population.
- A wide interval means high uncertainty; a narrow interval means the statistic looks more stable.

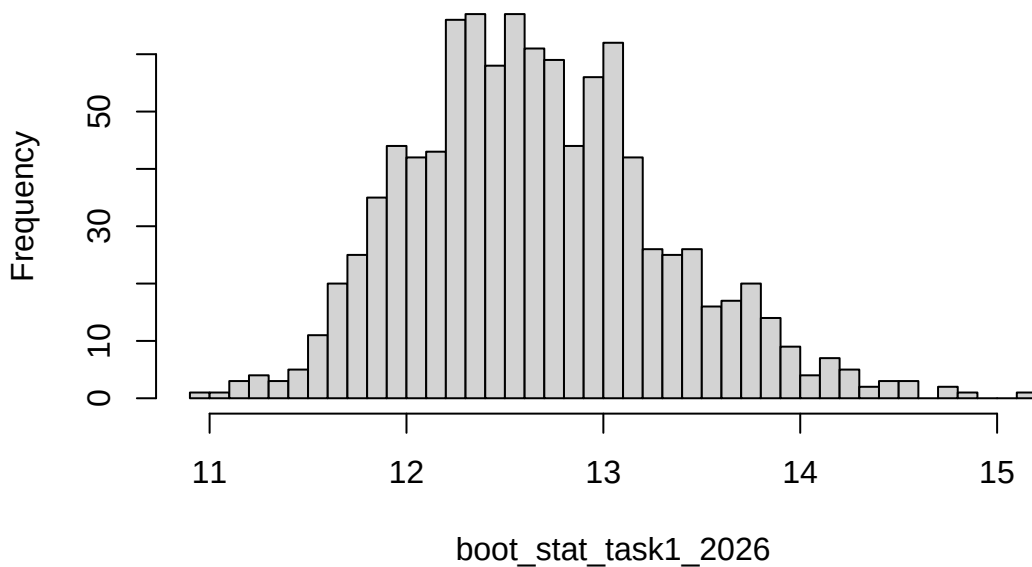
6.4.10 Bootstrap a variance or mean

```
# Set bootstrap parameters for the live Task 1 response. # B should be fairly large so the estimated s
set.seed(1) # Makes the bootstrap reproducible in the exam write-up.
B_task1_2026 <- 1000 # Number of bootstrap resamples.
n_task1_2026 <- length(y_task1_full_2026) # Original sample size in the live Task 1 response.
boot_stat_task1_2026 <- numeric(B_task1_2026) # Storage for one bootstrap statistic per iteration.

# Run the bootstrap loop. # The tell is sample(..., replace = TRUE).
for (b in seq_len(B_task1_2026)) { # Repeat B times.
  y_boot_task1_2026 <- sample(y_task1_full_2026, size = n_task1_2026, replace = TRUE) # Draw a fake sa
  boot_stat_task1_2026[b] <- var(y_boot_task1_2026) # Replace var() with mean(), coef(), or another st
}

# Plot the bootstrap distribution. # BAN404 has explicitly asked for histograms of bootstrap results.
hist(boot_stat_task1_2026, breaks = 40) # Visualizes the estimated sampling distribution.
```

Histogram of boot_stat_task1_2026



```
# Compute two common 95 percent confidence intervals. # Use both when the exam asks you to compare met
ci_percentile_task1_2026 <- quantile(boot_stat_task1_2026, c(0.025, 0.975)) # Empirical perce
ci_normal_task1_2026 <- mean(boot_stat_task1_2026) + c(-1, 1) * 1.96 * sd(boot_stat_task1_2026) # Norm
```

6.4.11 Bootstrap a regression coefficient

```
# Bootstrap a slope coefficient by resampling rows. # We must resample whole rows so x and y stay link
set.seed(1) # Reproducible bootstrap.
B <- 1000 # Number of bootstrap samples.
coef_boot <- numeric(B) # Storage for the bootstrapped slope estimates.

for (b in seq_len(B)) { # Repeat B times.
  idx <- sample(seq_len(nrow(df)), replace = TRUE) # Resample row indices, not single columns.
  fake_data <- df[idx, ] # Build a bootstrap dataset.
  fit_b <- lm(y ~ x, data = fake_data) # Refit the regression on the bootstrap sample.
  coef_boot[b] <- coef(fit_b)[2] # Save the slope coefficient of interest.
}
```

Error in seq_len(nrow(df)): argument must be coercible to non-negative integer

```
boot_se <- sd(coef_boot) # Bootstrap standard error is the standard deviation of t
```

6.4.12 Past exam links

- 2021 Task 1a–e
- 2023 Task 2b
- 2025 Task 1d

6.5 5. Ridge regression

6.5.1 What exam wording signals it?

- `sum((y - X%*%b1)^2) + la * sum(b1^2)`
- custom optimizer with `nlminb`
- question about demeaning and `lambda`

6.5.2 When to use it

Use ridge when the response is continuous, the OLS baseline is reasonable, and you want to reduce variance from many or correlated predictors.

6.5.3 Response type

Usually continuous regression in BAN404.

6.5.4 Core theory in easy words

Ridge is OLS plus a penalty on large coefficients. It shrinks coefficients toward zero to reduce variance.

6.5.5 Assumptions and weaknesses

- The response is usually **continuous**.
- Ridge still starts from a basically **linear mean structure**.
- It shrinks coefficients but usually does **not** set them exactly to zero, so it is less interpretable than sparse methods like LASSO.

6.5.6 What to tune

- The main tuning parameter is **lambda**.
- You may also need to decide whether predictors should be **centered or standardized**, depending on the exact implementation.

6.5.7 How to evaluate

- Compare ridge against OLS using the **same regression metric**, usually test MSE or CV MSE.
- If the question is theoretical, also explain what happens at **lambda = 0** and at large **lambda**.

6.5.8 BAN404-style custom ridge template

```
# Define the ridge objective function. # This is the exact 2025-style method tell.
ridge_objective_task1_2026 <- function(b1, X, y, la) {                # b1 = slope coefficients, la = lambda p
  sum((y - X %*% b1)^2) + la * sum(b1^2)                            # RSS plus L2 penalty.
}

# Define a ridge fitting helper. # Demeaning isolates the intercept so the penalty does not shrink it.
g_ridge_task1_2026 <- function(X, y, la) {                          # Inputs are predictor matrix, response
  q <- ncol(X)                                                       # Number of predictors.
  b0 <- mean(y)                                                       # Intercept becomes mean(y) after center
  yd <- y - b0                                                         # Center the response.
  Xd <- X                                                             # Copy X so we can center every predictor
  for (k in seq_len(q)) Xd[, k] <- X[, k] - mean(X[, k])             # Demean predictors column by column.
  b1start <- rep(0, q)                                                 # Start optimization at zero coefficient
  opt <- nlminb(b1start, ridge_objective_task1_2026, X = Xd, y = yd, la = la) # Numerically minimize t
```

```

    opt$par                                     # Return the estimated slope coefficient
  }

# Compare OLS, ridge with lambda = 0, and ridge with strong shrinkage on the live Task 1 data. # This
b_ols_task1_2026 <- coef(lm(y ~ ., data = task1_2026))[-1]                                # OLS
b_ridge0_task1_2026 <- g_ridge_task1_2026(X_task1_full_2026, y_task1_full_2026, la = 0)    # Ridge

```

Error: object 'X_task1_full_2026' not found

```

b_ridge1000_task1_2026 <- g_ridge_task1_2026(X_task1_full_2026, y_task1_full_2026, la = 1000) # Strong

```

Error: object 'X_task1_full_2026' not found

6.5.9 Common BAN404 written point

When `lambda = 0`, ridge becomes OLS. When `lambda` becomes large, coefficients are pulled toward zero, which increases bias a bit but often reduces variance a lot.

6.5.10 How to improve it

- Tune `lambda` with **LOOCV** or **k-fold CV** instead of guessing.
- If ridge helps but you also want variable selection, compare with **LASSO**.
- If the linear form itself is too rigid, compare with **GAM** or tree-based models instead of only shrinking coefficients.

6.5.11 How to interpret it

- Focus on the **shrinkage pattern** rather than classical p-values.
- The main written message is usually: ridge keeps all predictors but makes the coefficients smaller and often generalizes better than unstable OLS.

6.5.12 Past exam links

- 2025 Task 1a–c

6.6 6. LASSO

6.6.1 What exam wording signals it?

- absolute value penalty
- `glmnet`
- coefficient selection/shrinkage to zero

6.6.2 When to use it

Use LASSO when the response is continuous and you want both shrinkage and possible variable selection.

6.6.3 Response type

Usually continuous regression in BAN404.

6.6.4 Core theory in easy words

LASSO is like ridge, but the penalty is based on **absolute values**. This means it can set some coefficients exactly to zero and therefore perform variable selection.

6.6.5 Assumptions and weaknesses

- LASSO still assumes the core predictive relationship is basically **linear**.
- It is sensitive to the scale of predictors, so standardization is usually important.
- When predictors are very correlated, LASSO may select one and drop another in a way that looks unstable.

6.6.6 What to tune

- The main tuning parameter is **lambda**.
- In `glmnet`, you should also be aware of **standardization**, because scale affects the penalty.

6.6.7 How to evaluate

- Use **CV error** to choose **lambda**.
- Then compare the final model with OLS or ridge using **test MSE** or the same regression metric.
- Also inspect how many coefficients became exactly zero.

6.6.8 Full runnable template

```
# Build design matrices first. # glmnet cannot use factors directly, so model.matrix creates dummies.
X_train_task1_2026 <- model.matrix(y ~ ., data = task1_train_2026)[, -1] # Numeric matrix of predictor
y_train_task1_2026 <- task1_train_2026$y # Response vector.
X_test_task1_2026 <- model.matrix(y ~ ., data = task1_test_2026)[, -1] # Matching predictor matrix

# Tune lambda with cross-validation. # alpha = 1 means LASSO rather than ridge.
cv_lasso_task1_2026 <- cv.glmnet(X_train_task1_2026, y_train_task1_2026, alpha = 1, standardize = TRUE)

# Fit final LASSO at the best lambda. # lambda.min is the tuning value with the lowest CV error.
lasso_fit_task1_2026 <- glmnet(X_train_task1_2026, y_train_task1_2026, alpha = 1, lambda = cv_lasso_tas

# Predict on test data. # We evaluate it the same way as other regression models.
y_pred_lasso_task1_2026 <- as.numeric(predict(lasso_fit_task1_2026, newx = X_test_task1_2026)) # Conve

# Score with MSE. # Same metric as OLS or ridge for fair comparison.
mse_lasso_task1_2026 <- mean((task1_test_2026$y - y_pred_lasso_task1_2026)^2) # Smaller is better.

# Inspect coefficients. # Exact zeros are the key LASSO signature.
coef(lasso_fit_task1_2026) # Variables with zero coefficients are effectively removed.
```

```
7 x 1 sparse Matrix of class "dgCMatrix"
      s0
(Intercept) 3.48772812
x1          1.04630177
x2          0.04226472
x3         -1.62090273
x4         -0.02494538
x5          1.98939335
x6          0.02908744
```

6.6.9 How to improve it

- Tune `lambda` with **CV** instead of guessing.
- Compare with **ridge** if predictors are highly correlated and you do not want aggressive selection.
- Compare with **GAM** or tree-based models if the bigger problem is nonlinearity rather than too many linear predictors.

6.6.10 How to interpret it

- The key interpretation point is that some predictors may be shrunk all the way to **zero**, which makes the model simpler.
- If the exam asks “which variables matter most,” the nonzero coefficients give a first answer, but always connect them back to the business context.

6.6.11 Past exam links

- 2023 Task 1d–e
-

6.7 7. GAM / smoothing splines / backfitting

6.7.1 What exam wording signals it?

- “There might be non-linear relationships”
- `s(x)`
- `smooth.spline`
- backfitting explanation

6.7.2 When to use it

Use GAM when a continuous response still suggests a regression problem, but the straight-line OLS assumption looks too restrictive for some predictors.

6.7.3 Response type

Usually continuous regression in BAN404.

6.7.4 Core theory in easy words

A GAM lets some predictors stay linear while other predictors are allowed to bend smoothly. It is a good upgrade when the OLS straight-line assumption is too rigid.

6.7.5 Assumptions and weaknesses

- The response is usually **continuous** in BAN404’s GAM questions.
- Smooth terms help with **nonlinearity**, but GAM is still more structured and less automatic than a fully flexible tree ensemble.
- Too much smoothing can underfit; too much wiggleness can overfit.

6.7.6 What to tune

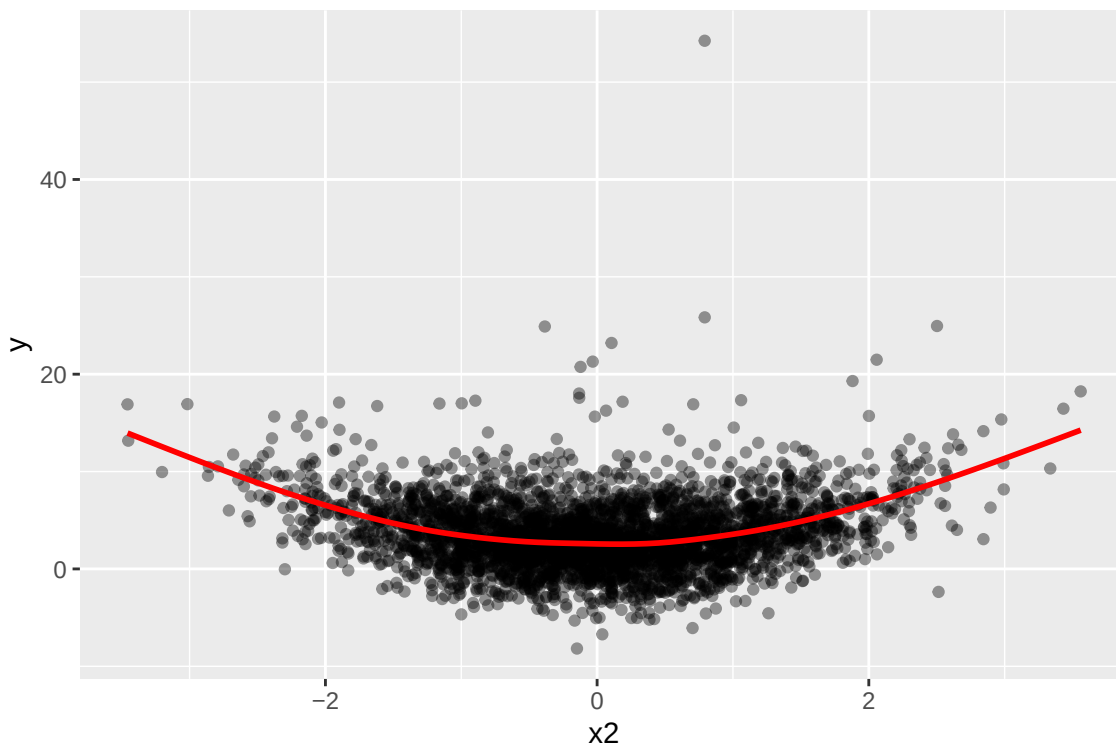
- Which predictors should be modeled with **smooth terms**.
- In some settings, the **smoothness / effective degrees of freedom**.
- The practical BAN404 choice is often simply: **which variable gets `s(x)`**.

6.7.7 How to evaluate

- Compare GAM with OLS using the **same regression metric**, often MSE or explained variation.
- Also evaluate whether the smooth plot looks substantively meaningful rather than adding wiggles for no reason.

6.7.8 Full runnable template

```
# Plot y against a candidate predictor from the live Task 1 data first. # BAN404 often expects you to
task1_train_2026 %>%
  ggplot(aes(x = x2, y = y)) +
  geom_point(alpha = 0.4) +
  geom_smooth(se = FALSE, color = "red") # Curvature in the smooth suggests a non-linear relationship.
```

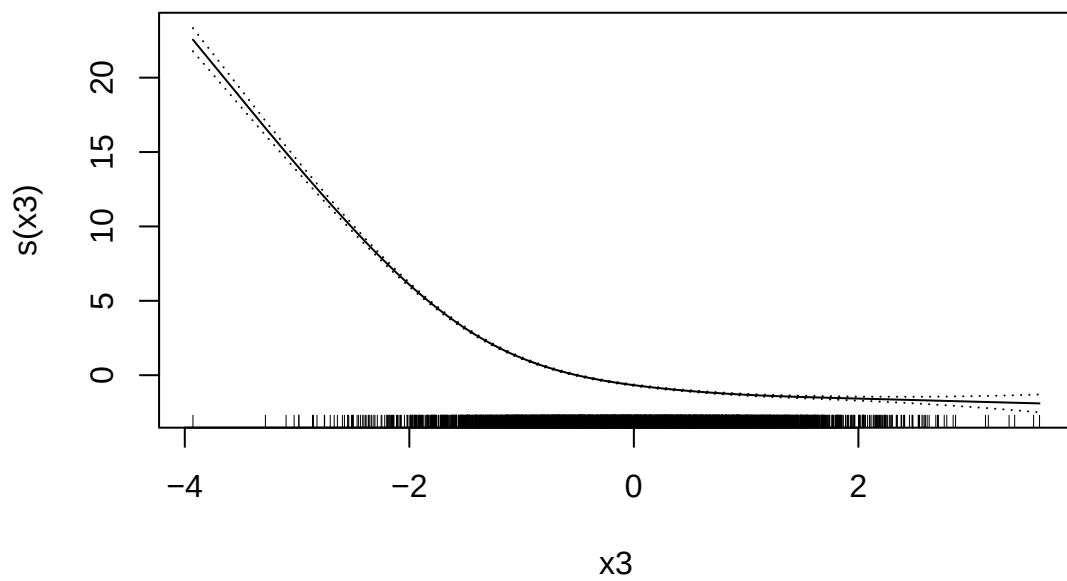
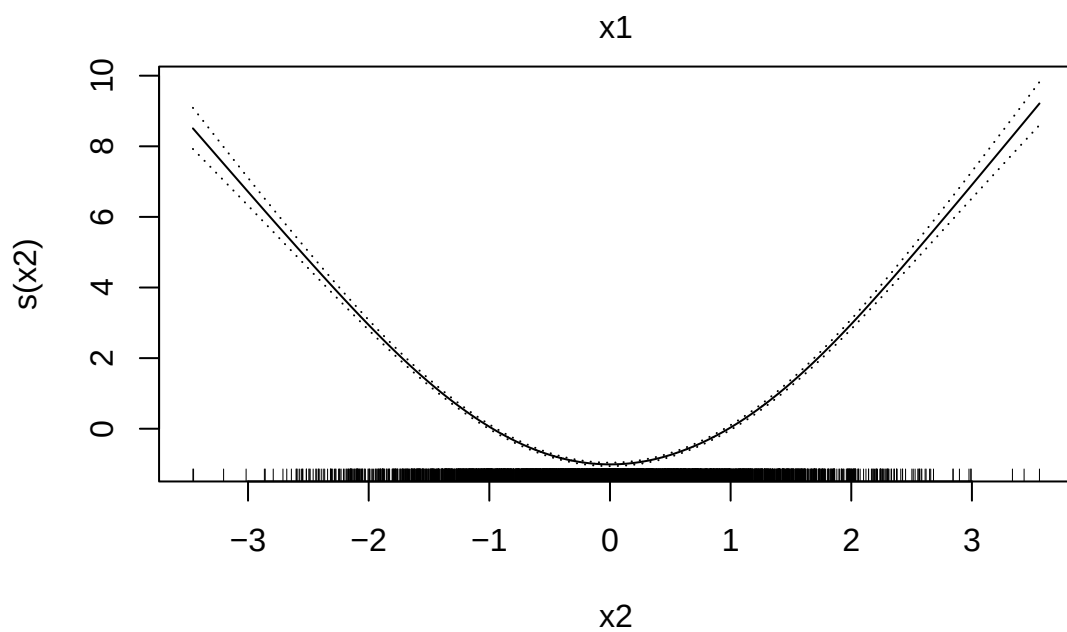
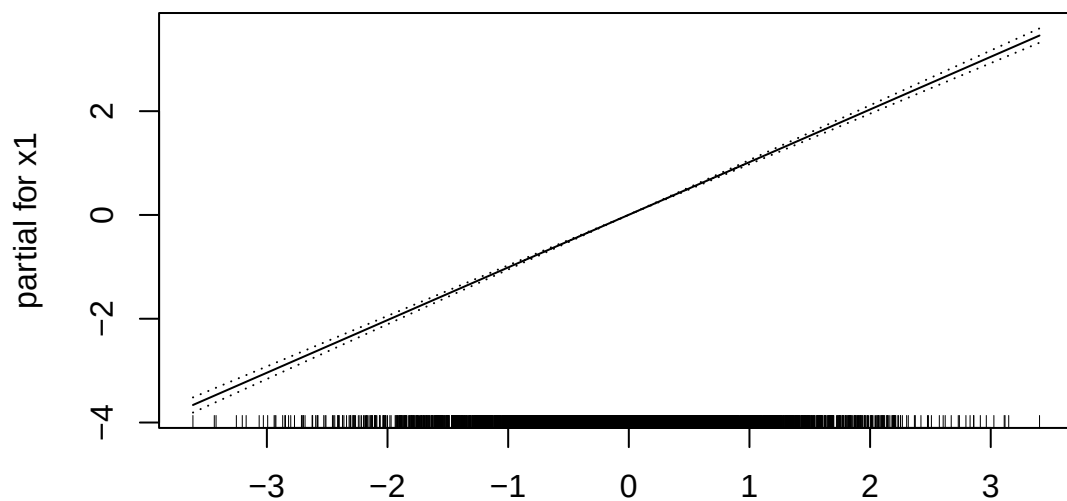


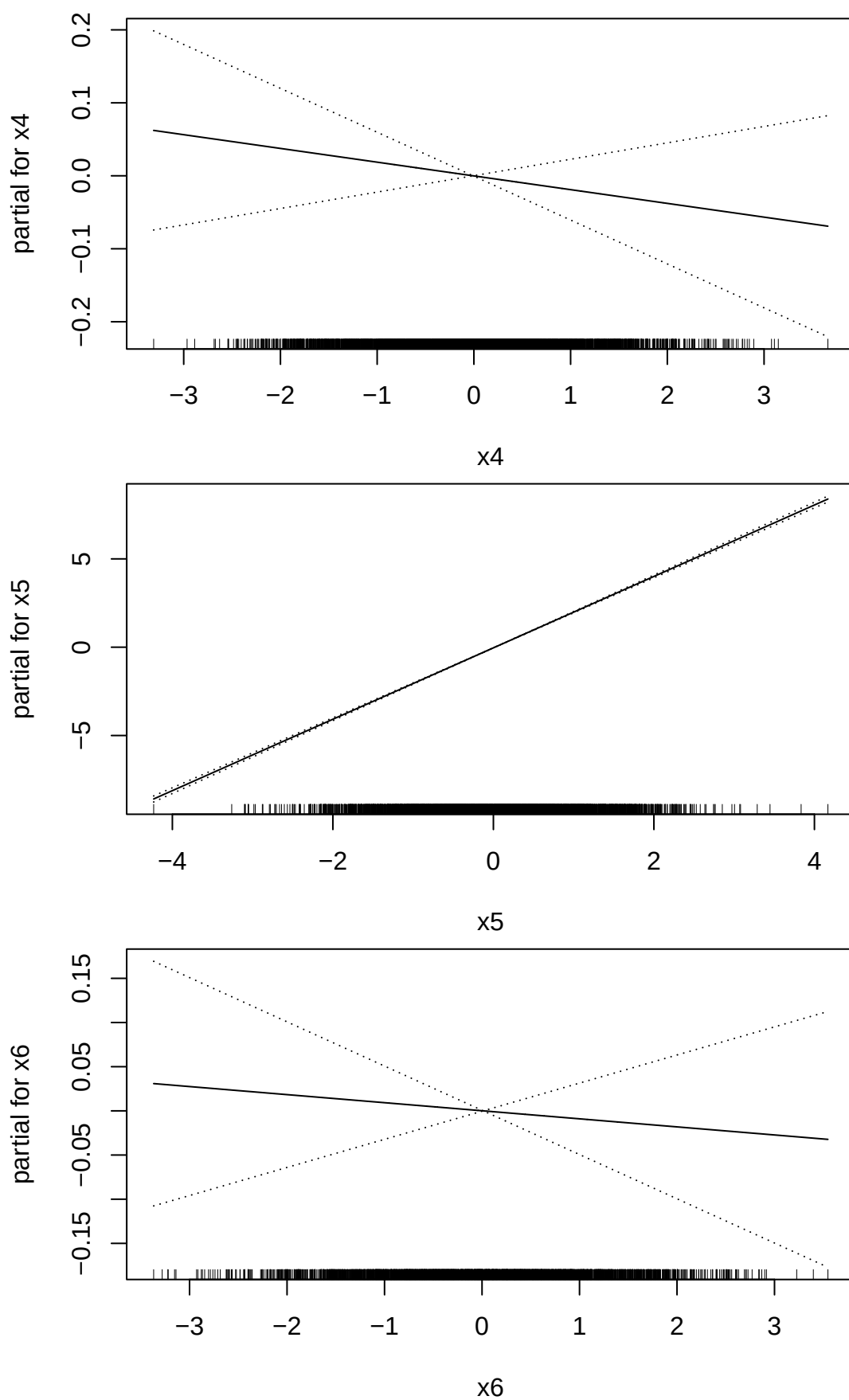
```
# Fit the OLS baseline. # We need a benchmark before claiming GAM improves anything.
ols_task1_2026 <- lm(y ~ ., data = task1_train_2026) # Linear benchmark.
mse_ols_task1_2026 <- mean(residuals(ols_task1_2026)^2) # In-sample MSE for baseline comparison.

# Fit a GAM with smooth terms that are plausible for the live Task 1 data. # x2 and x3 are natural fir
gam_task1_2026 <- gam(y ~ x1 + s(x2) + s(x3) + x4 + x5 + x6, data = task1_train_2026) # Let x2 and x3
mse_gam_task1_2026 <- mean(residuals(gam_task1_2026)^2) # In-sample MS

# Compute explained variation from MSE if asked. # This mirrors the 2025 Task 1e style calculation.
var_y_task1_2026 <- mean((task1_train_2026$y - mean(task1_train_2026$y))^2) # Total variation benchmark
r2_ols_task1_2026 <- 1 - mse_ols_task1_2026 / var_y_task1_2026 # Share explained by OLS.
r2_gam_task1_2026 <- 1 - mse_gam_task1_2026 / var_y_task1_2026 # Share explained by GAM.

# Plot the smooth term. # This gives a visual interpretation of the non-linear effect.
plot(gam_task1_2026, se = TRUE) # Shows the estimated smooth relationship.
```





6.7.9 Backfitting in plain English

Backfitting updates one smooth term at a time by fitting it to the part of y not currently explained by the other smooth terms. Then it cycles until the updates stop changing much.

6.7.10 How to improve it

- Start from **OLS**, then only add smooth terms where plots suggest curvature.
- Compare one-smooth GAM with a larger GAM rather than making everything smooth immediately.
- If a GAM still misses strong interactions or complex structure, compare with **random forest** or **boosting**.

6.7.11 How to interpret it

- Interpret the **shape** of the smooth term: increasing, decreasing, U-shaped, flattening, and so on.
- The main story is usually not a single slope coefficient, but whether the relationship is clearly **nonlinear** and whether that improves fit over OLS.

6.7.12 Past exam links

- 2021 Task 2d–e
 - 2022 Task 2e–f
 - 2024 Task 1e theory
 - 2025 Task 1e
-

6.8 8. KNN / local regression

6.8.1 What exam wording signals it?

- distance calculation `d <- abs(x - x0)`
- `order(d)[1:K]`
- local fit using nearest neighbors

6.8.2 When to use it

Use KNN/local regression when the exam asks for a local prediction rule driven by **nearby observations** rather than one global line.

6.8.3 Response type

Usually continuous regression in BAN404's local regression tasks.

6.8.4 Core theory in easy words

KNN/local regression predicts a point using the observations closest to it. Small K means very flexible; large K means smoother but more biased.

6.8.5 Assumptions and weaknesses

- Nearby observations should carry similar information about the response.
- Distance matters a lot, so **scale differences** across predictors can distort the method.
- Small K can overfit badly; large K can oversmooth and miss local structure.

6.8.6 What to tune

- The key tuning parameter is K , the number of neighbors.
- In multivariate settings you may also need to think about **distance definition** and **standardization**.

6.8.7 One-predictor local regression template

```
# Define the BAN404 2024-style local regression function. # This is KNN-like because it uses nearest neighbors.
f_local <- function(x0, x, y, K = 3) {
  d <- abs(x - x0) # x0 is the point we want to predict at.
  o <- order(d)[1:K] # Distance from every observed x to the target x0.
  local_df <- tibble(x = x[o], y = y[o]) # Indices of the K nearest observations.
  reg <- lm(y ~ x, data = local_df) # Small local dataset of the nearest neighbors.
  predict(reg, newdata = tibble(x = x0)) # Local linear regression on only those neighbors.
} # Predict the response at x0.
```

6.8.8 Tune K with LOOCV

```
# Define LOOCV for the local regression rule. # 2024 asked for optimal K using leave-one-out CV.
loo_local <- function(K, x, y) {
  pred <- numeric(length(y)) # One held-out prediction per observation.
  for (i in seq_along(y)) {
    pred[i] <- f_local(x0 = x[i], x = x[-i], y = y[-i], K = K) # Leave out i, fit locally on the rest.
  }
  mean((y - pred)^2) # Average left-out squared error.
}

K_grid_task1_2026 <- 2:10 # Candidate neighbors
cv_mse_task1_2026 <- sapply(K_grid_task1_2026, function(k) loo_local(k, task1_2026$x2, task1_2026$y))
K_best_task1_2026 <- K_grid_task1_2026[which.min(cv_mse_task1_2026)] # Best K minimizes MSE
```

6.8.9 Multi-predictor note

When there are several predictors, replace the 1D absolute distance with a multivariate distance such as Euclidean distance. Standardization becomes important because large-scale variables otherwise dominate the distance.

6.8.10 How to evaluate

- Use **LOOCV** or CV to choose K.
- Compare candidate K values using **CV MSE** for regression.
- Final predictive comparison should still be made on the same test set as other regression models if the task includes a final model comparison.

6.8.11 How to improve it

- Tune K instead of choosing it arbitrarily.
- Standardize predictors in multi-predictor distance problems.
- If the local method is too unstable, compare with **GAM** or tree-based regression models.

6.8.12 How to interpret it

- Small K means the fit is **very local and flexible**.
- Large K means the fit is **smoother and more biased**.
- In written answers, the main interpretation is usually the **bias-variance tradeoff** rather than a coefficient story.

6.8.13 Past exam links

- 2024 Task 1a–e
-

6.9 9. Regression tree

6.9.1 What exam wording signals it?

- continuous target
- tree plot / branch interpretation
- compare to OLS or RF using MSE

6.9.2 When to use it

Use a regression tree when the response is continuous and you suspect important **cutoffs, interactions, or nonlinear splits**.

6.9.3 Response type

Continuous regression outcome.

6.9.4 Core theory in easy words

A regression tree repeatedly splits the predictor space into regions and predicts the average y inside each terminal region.

6.9.5 Assumptions and weaknesses

- Trees do not need linearity, but a single tree can be **unstable**.
- Small changes in the data can change the split structure a lot.
- A single regression tree often predicts worse than bagging or random forest, even if it is easy to explain.

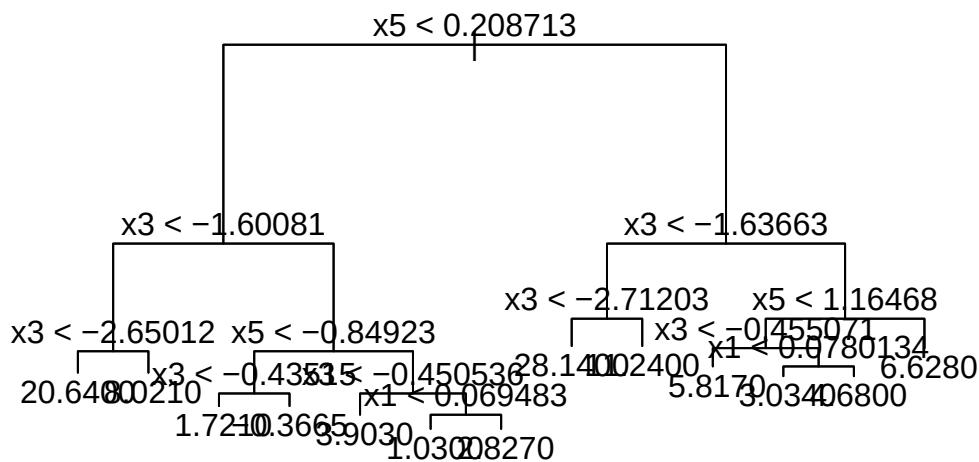
6.9.6 What to tune

- In simple BAN404 workflows, the practical tuning choices are tree size / complexity and whether to prune.
- Even when no explicit pruning is shown, the exam may still ask whether a larger ensemble would improve stability.

6.9.7 Full runnable template

```
# Fit a regression tree on the live Task 1 data. # Trees split the predictor space into boxes and predict
tree_reg_task1_2026 <- tree(y ~ ., data = task1_train_2026) # Continuous response automatically means

# Plot the tree. # BAN404 often expects a plot plus interpretation of a branch.
plot(tree_reg_task1_2026) # Draws the split structure.
text(tree_reg_task1_2026, pretty = 0) # Prints split labels and terminal node predictions.
```



```
# Predict on test data. # Same honest evaluation logic as OLS.
y_pred_tree_task1_2026 <- predict(tree_reg_task1_2026, newdata = task1_test_2026) # Continuous prediction

# Evaluate with MSE. # Continuous response means regression metric.
mse_tree_task1_2026 <- mean((task1_test_2026$y - y_pred_tree_task1_2026)^2) # Lower is better.
```

6.9.8 How to evaluate

- Use **test MSE** for continuous outcomes.
- If the exam gives both OLS and tree, compare them on the **same split** and the **same error metric**.

6.9.9 How to improve it

- If the single tree is unstable or weak, move to **bagging** or **random forest**.
- If the tree is too rough for the task, compare it with **GAM** or another smoother regression method.

6.9.10 How to interpret it

- Interpret each branch as a sequence of **if-then split rules**.
- Each terminal node prediction is the mean response for observations landing in that region.
- Trees are especially good when the exam asks you to explain **segments** rather than coefficients.

6.9.11 Past exam links

- 2023 Task 1f–g

6.10 10. Classification tree

6.10.1 What exam wording signals it?

- binary/categorical target
- plot and interpret a branch
- use threshold or direct class prediction

6.10.2 When to use it

Use a classification tree when the response is categorical and you want a split-based classifier that is easy to visualize.

6.10.3 Response type

Binary or multiclass classification outcome.

6.10.4 Core theory in easy words

A classification tree repeatedly splits the predictor space into regions that become as pure as possible with respect to the class labels.

6.10.5 Assumptions and weaknesses

- Trees do not assume linearity or normal predictors.
- A single classification tree can be easy to explain but often **unstable** and weaker than an ensemble.
- Threshold choice can still matter when you convert predicted class probabilities rather than using the direct class output.

6.10.6 What to tune

- Tree size / complexity and any probability threshold if you use predicted probabilities.
- In BAN404, the most exam-relevant practical choice is often whether to compare the single tree with **random forest**.

6.10.7 Full runnable template

```
# Fit a classification tree on the live customer churn data. # This is the tree analogue of logistic r
tree_cls_customer_2026 <- tree(churn ~ ., data = customer_train_2026) # churn is already a factor for
```

```
Error in eval(expr, p): object 'customer_train_2026' not found
```

```
# Plot and label the tree. # Exam answers look stronger when you show the rules visually.
plot(tree_cls_customer_2026) # Tree structure.
```

```
Error: object 'tree_cls_customer_2026' not found
```

```
text(tree_cls_customer_2026, pretty = 0) # Split labels and terminal nodes.
```

```
Error: object 'tree_cls_customer_2026' not found
```

```
# Get predicted probabilities. # Many BAN404 tasks want class predictions based on a threshold.
pred_prob_tree_customer_2026 <- predict(tree_cls_customer_2026, newdata = customer_test_2026) # For cl
```

```
Error: object 'tree_cls_customer_2026' not found
```

```
# Convert probabilities to class labels. # Use the probability of the positive class.
pred_class_tree_customer_2026 <- if_else(pred_prob_tree_customer_2026[, 2] > 0.30, levels(customer_train_2026$churn),
  factor(levels = levels(customer_train_2026$churn)))
```

```
Error: object 'pred_prob_tree_customer_2026' not found
```

```
# Evaluate the classification tree. # Same metric style as logistic regression.
cm_tree_customer_2026 <- table(actual = customer_test_2026$churn, predicted = pred_class_tree_customer_2026)
```

```
Error: object 'customer_test_2026' not found
```

```
acc_tree_customer_2026 <- sum(diag(cm_tree_customer_2026)) / sum(cm_tree_customer_2026)
```

Error in h(simpleError(msg, call)): error in evaluating the argument 'x' in selecting a method for function

6.10.8 How to evaluate

- Use a **confusion matrix** and **accuracy**.
- If the class balance is uneven, also report **sensitivity** and **specificity** or at least discuss why accuracy may be incomplete.

6.10.9 How to improve it

- Compare the single tree with **random forest** or **boosting** if the question asks for better prediction.
- Tune the threshold if you are using probabilities and the positive class is rare.

6.10.10 How to interpret it

- Interpret each split as a decision rule.
- Interpret each leaf as the predicted class or dominant class profile for that segment.
- The strength of the model is interpretability by **rules**, not by coefficients.

6.10.11 Past exam links

- 2022 Task 1d–f
 - 2024 Task 2e
-

6.11 11. Bagging

6.11.1 What exam wording signals it?

- “bagging”
- “many bootstrap trees”
- “improve a single tree by averaging”

6.11.2 When to use it

Use bagging when a single tree is too unstable and you want to reduce variance by averaging many bootstrap trees.

6.11.3 Response type

Can be used for both regression and classification.

6.11.4 Core theory in easy words

Bagging means fitting many trees on bootstrap samples and averaging them. This reduces variance compared with a single unstable tree.

6.11.5 Assumptions and weaknesses

- Bagging inherits the tree advantage of not needing linearity.
- It is usually more stable than a single tree but less interpretable.
- Because every split sees all predictors, bagging usually helps less than random forest when tree correlation is a big problem.

6.11.6 What to tune

- The main practical choices are **n**tree and, in the randomForest approximation, setting **m**try = **p**.

6.11.7 Safe bagging approximation template

```
# Approximate bagging with randomForest by letting mtry equal all predictors. # This is the standard t
p_customer_2026 <- ncol(customer_train_2026) - 1 # Number
```

Error: object 'customer_train_2026' not found

```
bag_fit_customer_2026 <- randomForest(churn ~ ., data = customer_train_2026, mtry = p_customer_2026, nt
```

Error in eval(m\$data, parent.frame()): object 'customer_train_2026' not found

```
# Predict and evaluate. # Use MSE for regression or confusion matrix/accuracy for classification.
pred_bag_customer_2026 <- predict(bag_fit_customer_2026, newdata = customer_test_2026) # Average
```

Error: object 'bag_fit_customer_2026' not found

6.11.8 How to evaluate

- Use **MSE** for regression bagging and a **confusion matrix** / **accuracy** for classification bagging.
- Compare bagging with the corresponding **single tree** on the same split.

6.11.9 How to improve it

- Increase **n**tree if performance looks unstable.
- Compare with **random forest** if bagging still seems too correlated across trees.

6.11.10 How to interpret it

- Bagging is usually interpreted at the **model-comparison level**, not by individual splits.
- The key interpretation is that averaging many unstable trees often improves predictive stability.

6.11.11 Past exam links

- 2022 Task 2g–h
- conceptually related in later random forest tasks

6.12 12. Random forest

6.12.1 What exam wording signals it?

- explicit random forest request
- variable importance plot
- compare with logistic/tree/OLS

6.12.2 When to use it

Use random forest when you want a strong flexible model that can capture nonlinearities and interactions automatically.

6.12.3 Response type

Can be used for both regression and classification.

6.12.4 Core theory in easy words

Random forest is like bagging plus extra randomness in predictor selection. This decorrelates the trees and usually improves prediction further.

6.12.5 Assumptions and weaknesses

- Random forest does not need linearity or normal predictors.
- It is usually strong for prediction but weaker for clean coefficient-style interpretation.
- Variable importance helps, but it does not replace a full causal or explanatory story.

6.12.6 What to tune

- The main tuning choices are **ntree** and **mtry**.
- In exam answers, you can also frame the comparison itself as the main tuning/improvement step if the task does not demand a full search.

6.12.7 Classification template

```
# Fit random forest for classification on the live churn dataset. # RF is a high-frequency BAN404 impr
rf_cls_customer_2026 <- randomForest(churn ~ ., data = customer_train_2026, ntree = 300) # ntree contr
```

```
Error in eval(m$data, parent.frame()): object 'customer_train_2026' not found
```

```
# Predict classes on test data. # randomForest handles the voting internally.
pred_rf_cls_customer_2026 <- predict(rf_cls_customer_2026, newdata = customer_test_2026) # Default for
```

```
Error: object 'rf_cls_customer_2026' not found
```

```
# Evaluate. # Same classification scorecard as logistic and trees.
cm_rf_customer_2026 <- table(actual = customer_test_2026$churn, predicted = pred_rf_cls_customer_2026)
```

```
Error: object 'customer_test_2026' not found
```

```
acc_rf_customer_2026 <- sum(diag(cm_rf_customer_2026)) / sum(cm_rf_customer_2026)
```

```
Error in h(simpleError(msg, call)): error in evaluating the argument 'x' in selecting a method for func
```

```
# Inspect variable importance. # BAN404 often asks for plot plus interpretation.  
varImpPlot(rf_cls_customer_2026) # Visual importance summary.
```

```
Error: object 'rf_cls_customer_2026' not found
```

```
importance(rf_cls_customer_2026) # Numeric importance values.
```

```
Error: object 'rf_cls_customer_2026' not found
```

6.12.8 Regression template

```
# Fit random forest for regression on the live Task 1 data. # Same idea, but continuous y means averaging  
rf_reg_task1_2026 <- randomForest(y ~ ., data = task1_train_2026, ntree = 300) # RF regression often c  
  
# Predict on test data. # Honest out-of-sample comparison.  
y_pred_rf_task1_2026 <- predict(rf_reg_task1_2026, newdata = task1_test_2026)  
  
# Evaluate with MSE. # Same regression metric as other continuous-response models.  
mse_rf_task1_2026 <- mean((task1_test_2026$y - y_pred_rf_task1_2026)^2)
```

6.12.9 How to evaluate

- Use **MSE** for regression forests.
- Use **confusion matrix**, **accuracy**, and often **sensitivity/specificity** for classification forests.
- Compare the forest with the corresponding simpler baseline: OLS, logistic, or a single tree.

6.12.10 How to improve it

- Increase **ntree** or tune **mtry** if the question asks for model refinement.
- If the response is binary and rare, combine the forest with **threshold tuning** rather than relying only on default class output.
- If interpretability matters more than raw performance, fall back to a simpler baseline even if RF predicts slightly better.

6.12.11 How to interpret it

- Interpret the model mainly through **predictive performance** and **variable importance**.
- Avoid pretending the forest gives one clean coefficient per predictor; its strength is pattern capture, not a tidy linear equation.

6.12.12 Past exam links

- 2023 Task 1h-i and 2e
- 2024 Task 2f and 2k
- 2025 Task 2d

6.13 13. Boosting

6.13.1 What exam wording signals it?

- “boosted tree”
- `gbm`
- compare with random forest or logistic

6.13.2 When to use it

Use boosting when the exam asks for a sequential tree ensemble that may squeeze out more predictive power than a single tree.

6.13.3 Response type

Can be used for both regression and classification.

6.13.4 Core theory in easy words

Boosting builds trees **sequentially**, where each new tree tries to correct the mistakes of the earlier trees. Small learning rates make it learn more cautiously.

6.13.5 Assumptions and weaknesses

- Boosting is very flexible and does not rely on linearity.
- It can perform well, but it is less interpretable than simpler baseline models.
- If tuned too aggressively, it can overfit.

6.13.6 What to tune

- The main tuning choices are `n.trees`, `interaction.depth`, and `shrinkage`.
- For classification tasks, the `threshold` can also matter after probability prediction.

6.13.7 Classification template

```
# Convert churn to 0/1 if gbm needs numeric Bernoulli coding. # Some implementations work more smoothly
train_boost_customer_2026 <- customer_train_2026 %>% mutate(churn_num = if_else(churn == levels(churn)[2], 1, 0))
```

Error: object 'customer_train_2026' not found

```
test_boost_customer_2026 <- customer_test_2026 %>% mutate(churn_num = if_else(churn == levels(churn)[2], 1, 0))
```

Error: object 'customer_test_2026' not found

```
# Fit a boosted classification model. # distribution = "bernoulli" means binary classification.
boost_fit_customer_2026 <- gbm(
  churn_num ~ . - churn, # Use predictors to predict the numeric class target.
  data = train_boost_customer_2026, # Training data only.
  distribution = "bernoulli", # Binary outcome setup.
  n.trees = 1000, # Number of boosting steps.
  interaction.depth = 2, # Tree depth per boosting step.
  shrinkage = 0.01, # Learning rate; smaller means slower, often stabler learning.
  verbose = FALSE # Cleaner exam output.
)
```

```
Error in eval(mf, parent.frame()): object 'train_boost_customer_2026' not found
```

```
# Predict test probabilities. # Boosting naturally gives probabilities for Bernoulli tasks.
pred_prob_boost_customer_2026 <- predict(boost_fit_customer_2026, newdata = test_boost_customer_2026, n
```

```
Error: object 'boost_fit_customer_2026' not found
```

```
# Convert to classes at a lower threshold. # Same final classification logic as logistic regression, b
pred_class_boost_customer_2026 <- if_else(pred_prob_boost_customer_2026 > 0.30, levels(customer_train_2
  factor(levels = levels(customer_train_2026$churn)))
```

```
Error: object 'pred_prob_boost_customer_2026' not found
```

```
# Evaluate predictions. # Confusion matrix plus accuracy remains the safe BAN404 choice.
cm_boost_customer_2026 <- table(actual = customer_test_2026$churn, predicted = pred_class_boost_custome
```

```
Error: object 'customer_test_2026' not found
```

```
acc_boost_customer_2026 <- sum(diag(cm_boost_customer_2026)) / sum(cm_boost_customer_2026)
```

```
Error in h(simpleError(msg, call)): error in evaluating the argument 'x' in selecting a method for func
```

6.13.8 Regression template

```
# Fit a boosted regression tree on the live Task 1 data. # Gaussian distribution is the usual choice f
boost_reg_task1_2026 <- gbm(
  y ~ ., # Predict continuous y from the predictors.
  data = task1_train_2026, # Training data only.
  distribution = "gaussian", # Continuous-response setup.
  n.trees = 1000, # Number of boosting trees.
  interaction.depth = 2, # Controls how much interaction structure each tree can model.
  shrinkage = 0.01, # Small learning rate helps regularize the sequential fitting.
  verbose = FALSE # Cleaner output.
)

# Predict on test data. # Use the same number of trees chosen in the fitted model if not separately tu
y_pred_boost_task1_2026 <- predict(boost_reg_task1_2026, newdata = task1_test_2026, n.trees = 1000)

# Evaluate with MSE. # Same metric as other regression models.
mse_boost_task1_2026 <- mean((task1_test_2026$y - y_pred_boost_task1_2026)^2)
```

6.13.9 How to evaluate

- Use **MSE** for regression boosting.
- Use **confusion matrix**, **accuracy**, and where relevant **sensitivity/specificity** for classification boosting.
- Compare against simpler baselines such as logistic regression, a single tree, or random forest depending on the exam wording.

6.13.10 How to improve it

- Tune **n.trees**, **interaction.depth**, and **shrinkage** rather than fixing them blindly.
- Use a lower threshold in rare-event classification if 0.5 is unrealistic.
- If boosting does not clearly improve predictive performance, the safer exam recommendation may still be the simpler baseline.

6.13.11 How to interpret it

- Interpret boosting mainly through **comparative predictive performance**, not through one clean set of coefficients.
- The written takeaway is usually whether the sequential ensemble actually improved prediction enough to justify the extra complexity.

6.13.12 Past exam links

- 2021 Task 3d
 - 2025 Task 2e
-

7 Lower-probability fallback sections

7.1 LDA / QDA

Use when the question explicitly mentions discriminant analysis or when theory compares logistic regression with Gaussian class models.

```
# Fit LDA. # LDA assumes class-conditional normality with a shared covariance matrix.
lda_fit <- lda(y ~ ., data = train) # MASS::lda model.
```

Error in eval(expr, p): object 'train' not found

```
# Predict on test data. # Returns classes and posterior probabilities.
lda_pred <- predict(lda_fit, newdata = test)
```

Error: object 'lda_fit' not found

```
cm_lda <- table(actual = test$y, predicted = lda_pred$class)
```

Error: object 'test' not found

```
acc_lda <- sum(diag(cm_lda)) / sum(cm_lda)
```

Error in h(simpleError(msg, call)):error in evaluating the argument 'x' in selecting a method for function 'h'

7.2 SVM

Use if the question explicitly asks for support vector machines or tuning **cost** / **gamma**. Standardization is usually important.

```
# Fit an SVM classifier. # Radial kernel adds nonlinear flexibility.
svm_fit <- svm(y ~ ., data = train, kernel = "radial", cost = 1, gamma = 0.1) # e1071 workflow.
```

Error in eval(expr, p): object 'train' not found

```
pred_svm <- predict(svm_fit, newdata = test)
```

Error: object 'svm_fit' not found

```
cm_svm <- table(actual = test$y, predicted = pred_svm)
```

Error: object 'test' not found

```
acc_svm <- sum(diag(cm_svm)) / sum(cm_svm)
```

Error in h(simpleError(msg, call)): error in evaluating the argument 'x' in selecting a method for function 'h'

7.3 PCA / PCR

Use if the question asks to reduce many correlated predictors into a smaller number of components. Standardization is important when variable scales differ.

```
# Standardize predictors before PCA. # PCA is scale-sensitive, so large-scale variables otherwise dominate.
X_scaled <- scale(select(df, -y)) # Remove the response before unsupervised dimension reduction.
```

Error in select(df, -y): unused argument (-y)

```
# Run PCA. # prcomp is the standard R function.
pca_fit <- prcomp(X_scaled, center = TRUE, scale. = TRUE)
```

Error: object 'X_scaled' not found

```
# Inspect explained variance. # Scree plots help decide how many components to keep.
summary(pca_fit)
```

Error in h(simpleError(msg, call)): error in evaluating the argument 'object' in selecting a method for function 'h'

```
plot(pca_fit, type = "l")
```

Error: object 'pca_fit' not found

7.4 K-means

Use only if the question asks for clustering. Standardization is usually important.

```
# Standardize variables first. # Distance-based clustering is distorted by unequal scales.
X_scaled <- scale(df)
```

Error in as.vector(x, mode): cannot coerce type 'closure' to vector of type 'any'

```
# Fit k-means with a chosen number of clusters. # nstart > 1 reduces bad random starts.
km_fit <- kmeans(X_scaled, centers = 3, nstart = 20)
```

Error: object 'X_scaled' not found

```
# Inspect within-cluster sum of squares. # Helps when drawing an elbow plot over several K values.
km_fit$tot.withinss
```

Error: object 'km_fit' not found

8 Question wording → action map

If the exam says...	You should probably do...
“Explain what the following function is doing”	identify the tell, name the method, explain each line in plain English
“Find optimal lambda / K / threshold”	build a CV or LOOCV loop
“Predict new customers”	remove leakage/future-unavailable variables, split train/test, evaluate on test
“Which variables are associated with y?”	do EDA + fit interpretable model
“Can you improve predictions?”	compare baseline with flexible or ensemble model using same metric
“Interpret a branch”	describe the split rule and terminal prediction/class
“Show a 95% confidence interval”	bootstrap and report percentile and/or normal CI
“Why remove variable X?”	explain ID, leakage, or unrealistic future availability
“Find non-linear relationships”	scatterplots + GAM/smooth terms
“Compare model fit”	use same split and same metric

8.1 15 code lines that rescue you in BAN404

Code line	Why it matters
<code>glimpse(df)</code>	fastest way to see classes, names, and obvious import mistakes
<code>janitor::clean_names()</code>	makes ugly column names easier to use safely
<code>mutate(across(where(is.character), as.factor))</code>	converts character categories into model-ready factors
<code>select(-id)</code>	removes identifiers that usually do not generalize
<code>select(-antskad, -skadkost)</code>	classic leakage removal in 2025 insurance-style tasks
<code>set.seed(123)</code>	makes your split or resampling reproducible
<code>idx <- sample(seq_len(nrow(df)), size = floor(nrow(df) / 2))</code>	standard 50/50 train/test split pattern
<code>model.matrix(y ~ ., data = train)[, -1]</code>	creates dummy variables for <code>glmnet</code>
<code>lm(y ~ ., data = train)</code>	baseline regression model
<code>glm(y ~ ., data = train, family = binomial())</code>	baseline binary classification model
<code>predict(fit, newdata = test, type = "response")</code>	gets probabilities from logistic/boosting-style models
<code>table(actual = y_test, predicted = y_pred)</code>	standard confusion matrix pattern
<code>mean((y_test - y_pred)^2)</code>	standard regression error metric
<code>sample(y, size = length(y), replace = TRUE)</code>	the one-line tell for bootstrap resampling
<code>cv.glmnet(X_train, y_train, alpha = 1)</code>	safe LASSO tuning pattern

8.2 Base model → improvement model → interpretation guide

Model family	Safe base model	Natural improvement step	What to interpret first
continuous y with mostly linear story	OLS	ridge / LASSO	coefficients, test MSE, R-squared
continuous y with curvature	OLS	GAM	shape of the smooth term and fit improvement
continuous y with strong interactions	OLS	tree / random forest / boosting	test MSE and variable importance

Model family	Safe base model	Natural improvement step	What to interpret first
binary y with explanation goal	logistic regression	reduced logistic / threshold tuning	coefficient signs, odds, class balance
binary y with prediction goal	logistic regression	tree / random forest / boosting	confusion matrix, sensitivity, specificity
many correlated predictors	OLS or logistic	ridge / LASSO	shrinkage, stability, zeroed coefficients
unstable single tree	tree	bagging / random forest	whether predictive performance improves

9 Written answer templates

9.1 Why ridge differs from OLS

The ridge model differs from OLS because it adds an L2 penalty, $\lambda * \sum(\beta_j^2)$, to the residual sum of squares. When $\lambda = 0$, ridge reduces to OLS. When λ becomes larger, the coefficients are shrunk toward zero, which increases bias slightly but often reduces variance and improves generalization.

9.2 Why demeaning is needed in BAN404-style ridge

Demeaning is used so the intercept is handled separately and is not penalized. After centering y and each column of X, the intercept is simply the mean of y, while the optimization only needs to estimate the slope coefficients.

9.3 Why train/test is used

Because the goal is prediction for new observations, the model should be evaluated on unseen test data rather than on the training data it was fitted on. Training performance is usually too optimistic.

9.4 Why CV is used

Cross-validation is used to estimate test error while selecting a tuning parameter such as λ , K, or a classification threshold. It gives a more honest basis for model tuning than training error alone.

9.5 Why a variable is removed

I removed this variable because it would not realistically be available at prediction time / it directly contains information about the outcome / it is only an identifier and has no stable predictive meaning.

9.6 Why GAM helps

A GAM helps when the relationship between a predictor and the response is not well approximated by a straight line. By replacing a linear term with a smooth function, the model can capture systematic curvature and often reduce MSE.

9.7 How to interpret variable importance in random forest

The variable importance plot shows which predictors contribute most to reducing prediction error or node impurity across the forest. A higher importance value means the model relied more on that variable when building strong predictive trees.

10 2026 exam data reconnaissance — first pivot to the live datasets

This section is the **first live-data pivot** for the 2026 retake. The goal is to stop thinking only in generic old-exam terms and start thinking:

- what the two attached datasets look like,
- what they are already strongly hinting at,
- which BAN404 model families are most likely to matter tomorrow, and
- which answer structures are safest if the exam questions are short but demanding.

10.1 Fast map of the two live datasets

Dataset	File path if your working directory is Exam2026/	Rows $\times$ columns	Response type	First BAN404 instinct
task1_data.csv	task1_data.csv	5000 $\times$ 7	continuous y	Task 1 regression, regularization, nonlinearity checks, LOOCV/CV
customer_data.csv	customer_data.csv	2666 $\times$ 20	binary churn	Task 2 classification, threshold choice, leak-age/redundancy discussion, tree ensembles

10.2 Dataset 1 — task1_data.csv

10.2.1 What we already know before fitting anything advanced

- The file is **clean and numeric**: six predictors (**x1** to **x6**) and one continuous response (**y**).
- The predictors look approximately **standardized around 0** with similar spread, which is very friendly for regularization questions.
- A first-pass signal check suggests:
 - **x5** has a **strong positive** relationship with **y**,
 - **x3** has a **strong negative** relationship with **y**,
 - **x1** has a **moderate positive** relationship with **y**,

- x2 looks weak **linearly**, but stronger once we allow **nonlinear structure**,
- the predictors appear to be **almost uncorrelated with each other**, so heavy multicollinearity is not the first story here.
- A plain linear model should be a safe **baseline**, but the data already hints that a **nonlinear follow-up** may outperform it.

10.2.2 Why this matters for likely Task 1 wording

This dataset is already screaming three classic BAN404 directions:

1. **Baseline linear regression**
“Fit and interpret an OLS model.”
2. **Regularization / tuning**
“Compare OLS, ridge, and/or LASSO” or “choose `lambda` by CV/LOOCV.”
3. **Nonlinearity check**
“Motivate a GAM / local regression / more flexible model.”

10.2.3 First-pass working code for Task 1

```
# Read the live 2026 Task 1 dataset. # This assumes your R working directory is `Exam2026/`, where the
task1_2026 <- read_csv("task1_data.csv", show_col_types = FALSE) # Standard csv import because the fil

# Inspect the structure immediately. # We want to confirm that x1 to x6 and y are all numeric.
glimpse(task1_2026) # Fast structure check before we do any modeling.
```

```
Rows: 5,000
Columns: 7
$ x1 <dbl> 1.62434536, 1.74481176, -0.32241720, 0.04221375, 0.90085595, -0.691~
$ x2 <dbl> -0.61175641, -0.76120690, -0.38405435, 0.58281521, -0.68372786, -0.~
$ x3 <dbl> -0.52817175, 0.31903910, 1.13376944, -1.10061918, -0.12289023, -0.6~
$ x4 <dbl> -1.0729686, -0.2493704, -1.0998913, 1.1447237, -0.9357694, -0.84520~
$ x5 <dbl> 0.8654076, 1.4621079, -0.1724282, 0.9015907, -0.2678881, -0.6712461~
$ x6 <dbl> -2.30153870, -2.06014071, -0.87785842, 0.50249434, 0.53035547, -0.0~
$ y <dbl> 6.8471326, 5.9755284, 2.1372866, 5.9087997, 5.1735769, 3.7305469, 0~
```

```
# Build a quick correlation table with y. # This is a very fast exam move when you want to see which l
task1_linear_signal <- task1_2026 %>%
  summarise(across(starts_with("x"), ~ cor(.x, y))) %>% # Correlate each predictor with the continuous
  pivot_longer(cols = everything(), names_to = "variable", values_to = "cor_with_y") %>% # Turn the on
  arrange(desc(abs(cor_with_y))) # Largest absolute correlation first because it is most exam-useful.

task1_linear_signal # Print the table so we quickly see the strongest linear predictors.
```

```
# A tibble: 6 x 2
  variable cor_with_y
  <chr>      <dbl>
1 x5        0.561
2 x3       -0.446
3 x1        0.316
4 x2        0.0252
5 x4       -0.0142
6 x6        0.0114
```

```
# Screen for simple curvature. # Squared terms are a fast way to see whether a weak linear predictor m
task1_nonlinear_signal <- task1_2026 %>%
  transmute(
    x1_sq = x1^2, # Square x1 to check whether curvature matters more than the straight-line effect.
    x2_sq = x2^2, # x2 looked more promising after nonlinear screening than after plain linear screeni
    x3_sq = x3^2, # x3 already looks important linearly, but a squared term may show extra structure.
    x4_sq = x4^2, # Same logic for x4 even if the first signal is weaker.
    x5_sq = x5^2, # Strong linear predictor can still have extra curvature.
    x6_sq = x6^2, # Weak linear predictor may still become useful nonlinearly.
    y = y # Keep y in the same table so we can compute correlations in one tidy pipeline.
  ) %>%
  summarise(across(-y, ~ cor(.x, y))) %>% # Correlate each squared feature with y.
  pivot_longer(cols = everything(), names_to = "feature", values_to = "cor_with_y") %>% # Turn to long
  arrange(desc(abs(cor_with_y))) # Strongest nonlinear hints first.

task1_nonlinear_signal # This often tells us whether GAM/polynomial thinking is worth trying.
```

```
# A tibble: 6 x 2
  feature cor_with_y
  <chr>      <dbl>
1 x2_sq      0.390
2 x3_sq      0.308
3 x4_sq      0.100
4 x1_sq     -0.0161
5 x6_sq     -0.0111
6 x5_sq      0.00594
```

```
# Create a reproducible train/test split. # Predictive comparison should use unseen data, not only in-
set.seed(404) # Fixed seed gives a reproducible answer.
idx_task1_2026 <- sample(seq_len(nrow(task1_2026)), size = floor(0.7 * nrow(task1_2026))) # 70 percent
train_task1_2026 <- task1_2026[idx_task1_2026, ] # Training data for model fitting and tuning.
test_task1_2026 <- task1_2026[-idx_task1_2026, ] # Test data for honest final comparison.

# Fit the safest baseline model first. # OLS is the natural benchmark for a continuous response.
ols_task1_2026 <- lm(y ~ ., data = train_task1_2026) # Linear model using all six predictors.
summary(ols_task1_2026) # Coefficients, signs, and significance-style information.
```

Call:

```
lm(formula = y ~ ., data = train_task1_2026)
```

Residuals:

```
    Min       1Q   Median       3Q      Max
-4.831 -1.367 -0.364  0.946 42.881
```

Coefficients:

```
            Estimate Std. Error t value Pr(>|t|)
(Intercept)  3.48749    0.03834  90.955 <2e-16 ***
x1            1.05370    0.03803  27.705 <2e-16 ***
x2            0.04987    0.03839   1.299  0.194
x3           -1.62837    0.03799 -42.867 <2e-16 ***
x4           -0.03284    0.03883  -0.846  0.398
x5            1.99671    0.03801  52.537 <2e-16 ***
x6            0.03673    0.03849   0.954  0.340
---
```

```
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
Residual standard error: 2.268 on 3493 degrees of freedom
Multiple R-squared:  0.6104,    Adjusted R-squared:  0.6097
F-statistic:  912 on 6 and 3493 DF,  p-value: < 2.2e-16
```

```

# Evaluate OLS on the held-out test data. # MSE is the standard BAN404 regression metric.
pred_ols_task1_2026 <- predict(ols_task1_2026, newdata = test_task1_2026) # Predict y on unseen rows.
mse_ols_task1_2026 <- mean((test_task1_2026$y - pred_ols_task1_2026)^2) # Test MSE for the OLS baseline

# Fit a richer but still interpretable GAM. # We only smooth the variables that looked most suspicious
gam_task1_2026 <- gam(
  y ~ x1 + s(x2) + s(x3) + x4 + x5 + x6, # Keep a mostly simple model, but let x2 and x3 bend if the data suggests
  data = train_task1_2026
) # GAM is a natural "improvement" model when the linearity assumption may be too restrictive.

# Evaluate the GAM on the same test set. # Fair comparison means same split and same metric.
pred_gam_task1_2026 <- predict(gam_task1_2026, newdata = test_task1_2026) # Out-of-sample GAM prediction
mse_gam_task1_2026 <- mean((test_task1_2026$y - pred_gam_task1_2026)^2) # Test MSE for the flexible model

# Prepare design matrices for ridge or LASSO. # The predictors are already numeric, so this is straightforward
X_train_task1_2026 <- model.matrix(y ~ ., data = train_task1_2026)[, -1] # Remove the intercept column
y_train_task1_2026 <- train_task1_2026$y # Numeric response vector for training
X_test_task1_2026 <- model.matrix(y ~ ., data = test_task1_2026)[, -1] # Matching test matrix.

# Tune ridge by CV. # alpha = 0 means ridge rather than LASSO.
cv_ridge_task1_2026 <- cv.glmnet(X_train_task1_2026, y_train_task1_2026, alpha = 0, standardize = TRUE)
pred_ridge_task1_2026 <- as.numeric(predict(cv_ridge_task1_2026, newx = X_test_task1_2026, s = "lambda.1se"))
mse_ridge_task1_2026 <- mean((test_task1_2026$y - pred_ridge_task1_2026)^2) # Ridge test MSE.

# Tune LASSO by CV. # alpha = 1 means LASSO and may zero weak coefficients.
cv_lasso_task1_2026 <- cv.glmnet(X_train_task1_2026, y_train_task1_2026, alpha = 1, standardize = TRUE)
pred_lasso_task1_2026 <- as.numeric(predict(cv_lasso_task1_2026, newx = X_test_task1_2026, s = "lambda.1se"))
mse_lasso_task1_2026 <- mean((test_task1_2026$y - pred_lasso_task1_2026)^2) # LASSO test MSE.

# Collect the main predictive comparison in one table. # This is the safest exam layout when asked which model is best
tibble(
  model = c("OLS", "GAM", "Ridge", "LASSO"), # Names of the competing models.
  test_mse = c(mse_ols_task1_2026, mse_gam_task1_2026, mse_ridge_task1_2026, mse_lasso_task1_2026) # Test MSE
) %>%
  arrange(test_mse) # Lowest test MSE first because lower is better.

```

```

# A tibble: 4 x 2
  model test_mse
  <chr>   <dbl>
1 GAM     1.13
2 Ridge   4.59
3 LASSO    4.59
4 OLS     4.60

```

10.2.4 Short written answer you can reuse for Task 1

I started with OLS because y is continuous and OLS is the correct BAN404 baseline. I then checked whether a more flexible model was justified. The first-pass signal suggested strong linear effects for x_1 , x_3 , and x_5 , but also possible curvature especially for x_2 and x_3 , so a GAM is a natural improvement candidate. Because the predictors are already numeric and on similar scales, ridge and LASSO are also easy and appropriate follow-up models when the question asks for regularization or tuning of λ . I compared all models on the same test set using test MSE so the performance comparison stayed fair.

10.3 Dataset 2 — customer_data.csv

10.3.1 What the dataset is already telling us

- This is a **binary churn classification** dataset.
- The target is **imbalanced**: roughly **14.5% churners** and **85.5% non-churners**, so raw accuracy alone can mislead.
- Several variables come in **near-duplicate charge/minute pairs**:
 - total_day_minutes and total_day_charge
 - total_eve_minutes and total_eve_charge
 - total_night_minutes and total_night_charge
 - total_intl_minutes and total_intl_charge

These are not leakage, but they are essentially **redundant pricing transforms** of the same usage information. In practice, it is cleaner to keep the **minutes** and drop the matching **charges**.

- Early signal checks suggest:
 - customer_service_calls is very important, especially from **4 calls and up**,
 - international_plan = "Yes" is strongly associated with churn,
 - voice_mail_plan = "Yes" is associated with lower churn,
 - total_day_minutes is one of the strongest continuous usage predictors.

10.3.2 Why this matters for likely Task 2 wording

This dataset strongly prepares you for three common BAN404 Task 2 question types:

1. **Explanatory classification**
“Which variables are associated with churn?”
2. **Operational predictive classification**
“Predict churn for new customers and evaluate the model.”
3. **Improvement / threshold / model comparison**
“Should we change the threshold?” or “Does random forest improve on logistic regression?”

10.3.3 First-pass working code for Task 2

```
# Read the live 2026 Task 2 dataset. # This assumes your R working directory is `Exam2026/`, where the
customer_2026_raw <- read_csv("customer_data.csv", show_col_types = FALSE) # Standard CSV import.

# Clean names and recode the key categorical variables. # This makes the data easier and safer to work
customer_2026 <- customer_2026_raw %>%
  janitor::clean_names() %>% # Convert spaces and capitals into tidy snake_case names.
  mutate(
    churn = factor(churn, levels = c(FALSE, TRUE), labels = c("no", "yes")), # Make the binary target
    international_plan = factor(international_plan), # Text yes/no should be treated as a category, no
    voice_mail_plan = factor(voice_mail_plan), # Same logic for voice mail plan.
    state = factor(state), # State is categorical.
    area_code = factor(area_code) # Area code is better treated as a category than
  )
```

```
# Check class balance immediately. # Class imbalance affects threshold choice and metric interpretation
customer_2026 %>%
  count(churn) %>%                                # Count churners and non-churners.
  mutate(prop = n / sum(n))                       # Add proportions so we see imbalance directly.

# A tibble: 2 x 3
  churn      n prop
  <fct> <int> <dbl>
1 no     2278 0.854
2 yes     388 0.146

# Check two high-signal categorical variables. # These quick grouped summaries often become useful writing
customer_2026 %>%
  group_by(international_plan) %>%                # Compare churn rates across international-plan groups.
  summarise(churn_rate = mean(churn == "yes"), n = n()) # Mean of a logical condition gives the class proportion.

# A tibble: 2 x 3
  international_plan churn_rate      n
  <fct>                <dbl> <int>
1 No                    0.113  2396
2 Yes                    0.437   270

customer_2026 %>%
  group_by(voice_mail_plan) %>%                  # Compare churn rates across voice-mail groups.
  summarise(churn_rate = mean(churn == "yes"), n = n()) # Same exam-friendly churn-rate summary.

# A tibble: 2 x 3
  voice_mail_plan churn_rate      n
  <fct>                <dbl> <int>
1 No                    0.167  1933
2 Yes                    0.0887  733

# Check whether customer service calls looks operationally important. # This variable often shows a strong relationship with churn.
customer_2026 %>%
  group_by(customer_service_calls) %>%          # Group customers by number of service calls.
  summarise(churn_rate = mean(churn == "yes"), n = n()) %>% # Compute the churn rate in each group.
  arrange(customer_service_calls)              # Keep the table in natural call-count order.

# A tibble: 10 x 3
  customer_service_calls churn_rate      n
  <dbl>                <dbl> <int>
1         0            0.142    555
2         1            0.105    945
3         2            0.102    608
4         3            0.106    348
5         4            0.481    133
6         5            0.592     49
7         6            0.588     17
8         7            0.625      8
9         8            1         1
10        9            1         2

# Remove redundant charge columns before modeling. # The charge columns are almost exact rescalings of usage.
customer_model_2026 <- customer_2026 %>%
  select(
    -total_day_charge, # Drop redundant day charge because total_day_minutes carries the same usage information.
    -total_eve_charge, # Drop redundant evening charge.
```

```

    -total_night_charge, # Drop redundant night charge.
    -total_intl_charge   # Drop redundant international charge.
  )

```

Error in select(., -total_day_charge, -total_eve_charge, -total_night_charge, : unused arguments (-total

```

# Create a reproducible train/test split. # Predictive evaluation should happen on unseen customers.
set.seed(404) # Fixed seed for reproducibility.
idx_customer_2026 <- sample(seq_len(nrow(customer_model_2026)), size = floor(0.7 * nrow(customer_model_2026)))

```

Error: object 'customer_model_2026' not found

```

train_customer_2026 <- customer_model_2026[idx_customer_2026, ] # Training customers.

```

Error: object 'customer_model_2026' not found

```

test_customer_2026 <- customer_model_2026[-idx_customer_2026, ] # Test customers.

```

Error: object 'customer_model_2026' not found

```

# Fit logistic regression first. # Logistic regression is the safest binary baseline because it is interpretable.
logit_customer_2026 <- glm(churn ~ ., data = train_customer_2026, family = binomial()) # Predict churn

```

Error in eval(mf, parent.frame()): object 'train_customer_2026' not found

```

summary(logit_customer_2026) # Coefficients and directions for explanatory discussion.

```

Error in h(simpleError(msg, call)): error in evaluating the argument 'object' in selecting a method for function 'h'

```

# Tune the classification threshold on training data. # We should not choose the threshold by looking at the training data.
prob_train_customer_2026 <- predict(logit_customer_2026, newdata = train_customer_2026, type = "response")

```

Error: object 'logit_customer_2026' not found

```

threshold_grid_customer_2026 <- seq(0.10, 0.50, by = 0.05) # Lower thresholds are worth checking because they are more conservative.

threshold_results_customer_2026 <- map_dfr(threshold_grid_customer_2026, function(th) { # Score each candidate threshold
  pred_train_th <- if_else(
    prob_train_customer_2026 > th, "yes", "no" # Convert probabilities to classes using the candidate threshold
  ) %>%
    factor(levels = c("no", "yes")) # Force the class order to stay stable.

  cm_train_th <- table(actual = train_customer_2026$churn, predicted = pred_train_th) # Confusion matrix
  sens_train_th <- cm_train_th["yes", "yes"] / sum(cm_train_th["yes", ]) # Sensitivity for "yes"
  spec_train_th <- cm_train_th["no", "no"] / sum(cm_train_th["no", ]) # Specificity for "no"

  tibble(
    threshold = th, # Candidate threshold we are testing.
    sensitivity = sens_train_th, # True-positive rate.
    specificity = spec_train_th, # True-negative rate.
    balanced_accuracy = mean(c(sens_train_th, spec_train_th)) # Safe summary metric under class imbalance
  )
})

```

```
Error in `map()``:  
i In index: 1.  
Caused by error in `.f()``:  
! object 'prob_train_customer_2026' not found
```

```
best_threshold_customer_2026 <- threshold_results_customer_2026 %>%  
  slice_max(order_by = balanced_accuracy, n = 1) %>% # Pick the threshold with the best balanced accuracy  
  pull(threshold) # Extract the threshold value itself.
```

```
Error: object 'threshold_results_customer_2026' not found
```

```
threshold_results_customer_2026 # Print the threshold comparison table.
```

```
Error: object 'threshold_results_customer_2026' not found
```

```
best_threshold_customer_2026 # Print the chosen threshold.
```

```
Error: object 'best_threshold_customer_2026' not found
```

```
# Evaluate logistic regression on the held-out test set. # This is the honest final score for the chosen threshold  
prob_test_customer_2026 <- predict(logit_customer_2026, newdata = test_customer_2026, type = "response")
```

```
Error: object 'logit_customer_2026' not found
```

```
pred_test_customer_2026 <- if_else(prob_test_customer_2026 > best_threshold_customer_2026, "yes", "no")  
  factor(levels = c("no", "yes")) # Final test-set class prediction.
```

```
Error: object 'prob_test_customer_2026' not found
```

```
cm_logit_customer_2026 <- table(actual = test_customer_2026$churn, predicted = pred_test_customer_2026)
```

```
Error: object 'test_customer_2026' not found
```

```
acc_logit_customer_2026 <- sum(diag(cm_logit_customer_2026)) / sum(cm_logit_customer_2026)
```

```
Error in h(simpleError(msg, call)): error in evaluating the argument 'x' in selecting a method for function 'diag'
```

```
sens_logit_customer_2026 <- cm_logit_customer_2026["yes", "yes"] / sum(cm_logit_customer_2026["yes", ], )
```

```
Error: object 'cm_logit_customer_2026' not found
```

```
spec_logit_customer_2026 <- cm_logit_customer_2026["no", "no"] / sum(cm_logit_customer_2026["no", ], )
```

```
Error: object 'cm_logit_customer_2026' not found
```

```
# Fit a random forest on the same cleaned training data. # This is the natural improvement model for n_estimators  
rf_customer_2026 <- randomForest(churn ~ ., data = train_customer_2026, ntree = 300) # Flexible ensemble
```

```
Error in eval(m$data, parent.frame()): object 'train_customer_2026' not found
```



```
pred_rf_customer_2026 <- predict(rf_customer_2026, newdata = test_customer_2026) # Default class
```

Error: object 'rf_customer_2026' not found

```
cm_rf_customer_2026 <- table(actual = test_customer_2026$churn, predicted = pred_rf_customer_2026) # R
```

Error: object 'test_customer_2026' not found

```
acc_rf_customer_2026 <- sum(diag(cm_rf_customer_2026)) / sum(cm_rf_customer_2026) # R
```

Error in h(simpleError(msg, call)): error in evaluating the argument 'x' in selecting a method for func

```
# Print the main scorecard. # This gives a compact answer block for a likely Task 2 comparison question
tibble(
  model = c("Logistic regression", "Random forest"), # Two likely comparison models.
  accuracy = c(acc_logit_customer_2026, acc_rf_customer_2026) # Same metric for both models.
)
```

Error: object 'acc_logit_customer_2026' not found

```
importance(rf_customer_2026) # Variable importance helps with interpretation if the exam asks what mat
```

Error: object 'rf_customer_2026' not found

10.3.4 Short written answer you can reuse for Task 2

I treated this as a binary churn classification problem. Because churn is the minority class, I did not rely on accuracy alone and instead tuned the classification threshold on the training data. I removed the charge variables because they are almost exact linear transforms of the corresponding minute variables and therefore add redundancy rather than genuinely new information. I started with logistic regression because it is the safest interpretable baseline, then compared it with random forest to check whether nonlinearities and interactions improved predictive performance. The most important early warning signals in the raw data are international plan, customer service calls, and usage intensity, especially day minutes.

10.4 Three very likely live-data question scenarios

10.4.1 Scenario 1 — Task 1 asks for a safe baseline plus one improvement model

I would start with OLS because the response y is continuous and OLS is the correct baseline. I would then compare it with a more flexible model such as a GAM, because the first-pass screening suggests that at least some predictors may enter nonlinearly even if their plain linear correlation is modest. I would compare the models on the same test set using MSE and recommend the model with the lower test MSE.

10.4.2 Scenario 2 — Task 2 asks which variables are associated with churn

I would use logistic regression because the response is binary and the coefficient signs are interpretable. Before fitting the model, I would remove redundant charge variables so the explanation is cleaner and less distorted by near-duplicate predictors. From the descriptive analysis, international plan, customer service calls, and usage variables already look important, so I would expect them to remain central in the fitted model.

10.4.3 Scenario 3 — Task 2 asks for the best predictive model

I would split the data into training and test sets, fit logistic regression as a baseline, tune the threshold on the training set, and then compare it with a more flexible model such as random forest on the same test set. Because churn is relatively rare, I would report not only accuracy but also sensitivity and specificity or balanced accuracy. I would recommend the more complex model only if it gives a clear predictive improvement on unseen data.

10.5 RStudio snippets to save for plug-and-play tomorrow

Paste these into **RStudio** → **Tools** / **Global Options** → **Code** → **Edit Snippets** if you want fast insertable templates.

```
snippet ban404task1live
  task1_2026 <- readr::read_csv("task1_data.csv", show_col_types = FALSE)
  glimpse(task1_2026)
  set.seed(404)
  idx <- sample(seq_len(nrow(task1_2026)), size = floor(0.7 * nrow(task1_2026)))
  train <- task1_2026[idx, ]
  test <- task1_2026[-idx, ]
  ols_fit <- lm(y ~ ., data = train)
  pred <- predict(ols_fit, newdata = test)
  mean((test$y - pred)^2)

snippet ban404task2live
  customer_2026 <- readr::read_csv("customer_data.csv", show_col_types = FALSE) %>%
    janitor::clean_names() %>%
    dplyr::mutate(
      churn = factor(churn, levels = c(FALSE, TRUE), labels = c("no", "yes")),
      international_plan = factor(international_plan),
      voice_mail_plan = factor(voice_mail_plan),
      state = factor(state),
      area_code = factor(area_code)
    ) %>%
    dplyr::select(-total_day_charge, -total_eve_charge, -total_night_charge, -total_intl_charge)

snippet ban404threshold
  threshold_grid <- seq(0.10, 0.50, by = 0.05)
  threshold_results <- purrr::map_dfr(threshold_grid, function(th) {
    pred_th <- if_else(prob_train > th, "yes", "no") %>% factor(levels = c("no", "yes"))
    cm_th <- table(actual = train$churn, predicted = pred_th)
    sens_th <- cm_th["yes", "yes"] / sum(cm_th["yes", ])
    spec_th <- cm_th["no", "no"] / sum(cm_th["no", ])
    tibble(threshold = th, balanced_accuracy = mean(c(sens_th, spec_th)))
  })
```

11 Dataset-anchored worked examples for likely exam questions

This section builds the generic method playbooks into **fuller, more exam-shaped examples** using datasets that are already known from BAN404. The point is not only to show code, but to show **why this exact code chunk is the right answer to this exact kind of exam wording**.

11.1 Fast map: which local dataset is useful for which model family?

Dataset	Repo-relative path from this file	Typical BAN404 use	Best methods to revise with it
data_task1.csv	../Previous Exam/2025/data_task1.csv	small numeric Task 1	OLS, ridge, LOOCV, bootstrap, GAM
dataOhlsson	package dataset via insuranceData	2025 Task 2 insurance claims	logistic, threshold choice, random forest, boosting
airline.csv	../Previous Exam/2024/airline.csv	2024 Task 2 two-perspective classification	descriptive analysis, logistic, tree, random forest
Churn.csv	../Previous Exam/2023/Churn.csv	2023 mixed regression + classification practice	OLS, LASSO, tree, random forest, logistic, bootstrap
OJ	package dataset via ISLR / ISLR2	2022 classification practice	logistic, classification tree, threshold CV

All file paths in this section are **relative to this compendium file inside Exam2026/**. So `../Previous Exam/2025/data_task1.csv` means “go up one folder from Exam2026/, then into Previous Exam/2025/”. Because the folder name contains a space, keep the whole path inside quotes exactly as shown in the code examples.

11.2 Example 1 — 2025 Task 1 style: data_task1.csv

11.2.1 Why this is a high-priority example

This is one of the cleanest BAN404 Task 1 datasets for practicing four recurring skills:

1. reading a weird file format correctly,
2. comparing **OLS** and **ridge**,
3. using **LOOCV** to tune a parameter, and
4. using **bootstrap** when the question is about **sampling uncertainty**, not prediction.

11.2.2 What kind of exam wording this example prepares you for

- “Extract X and y and fit OLS and ridge.”
- “Use LOOCV to choose `lambda`.”
- “Use bootstrap to estimate the sampling distribution of the variance.”
- “Look for non-linear relationships and motivate a GAM.”

11.2.3 Full worked code skeleton

```
# Read the 2025 Task 1 data using read_csv2. # We use read_csv2 because the file uses semicolon separator
task1_2025 <- read_csv2("../Previous Exam/2025/data_task1.csv", show_col_types = FALSE) # This is the

# Check that the import worked correctly. # We want y plus X1 to X20 as separate numeric columns, not
glimpse(task1_2025) # Always inspect structure before modeling so we catch file-format mistakes early.
```

```
Rows: 100
Columns: 21
$ y    <dbl> 2186.124917, 2085.589474, 3414.582206, -53.796121, 3295.945550, 27~
$ X1   <dbl> 0.3745881, 4.4061772, 7.9324674, -3.8582994, 0.6826511, -12.477686~
$ X2   <dbl> 10.6488074, 3.3884611, 0.2918325, 0.1175223, 13.9946012, -0.918975~
$ X3   <dbl> 13.1022462, 6.6017388, 7.5623799, 5.5517158, 2.2051861, 1.1716796,~
$ X4   <dbl> 1.57824839, 2.27929075, 9.14247999, -0.55921290, 9.00202263, 12.43~
```

```

$ X5 <dbl> 9.4504425, 1.7467380, 3.5327773, 19.8388088, 10.6563312, -6.989620~
$ X6 <dbl> 7.9000825, 6.4075095, 10.9601883, 1.8605896, -6.1491591, 8.9700713~
$ X7 <dbl> -0.39147703, 11.39105932, 18.71102647, 6.48661045, 5.49756697, -0.~
$ X8 <dbl> 13.982907, 7.078901, 6.213033, 6.270963, -7.163820, 4.435673, 20.5~
$ X9 <dbl> 9.2723345, 12.2772060, 27.0284552, -0.9354501, 17.5940930, 16.3767~
$ X10 <dbl> 4.202042, 8.760831, 8.991811, 8.382030, 5.040624, 10.033578, -6.17~
$ X11 <dbl> 2.7038402, 12.1029694, 4.4738981, 21.6014277, 11.5701730, 4.227252~
$ X12 <dbl> 2.539236, 11.811625, 12.996664, 4.404045, 4.210103, 17.636502, 22.~
$ X13 <dbl> 4.705819, 13.874440, 10.998682, 13.798241, 11.554997, 19.339477, 1~
$ X14 <dbl> 10.405978, 16.075364, 2.002804, 10.466786, 13.970140, 13.948130, 7~
$ X15 <dbl> 16.608143, 17.552993, 13.695066, 12.632006, 16.388130, 3.794391, 1~
$ X16 <dbl> 2.4756213, -0.8051675, 23.7743478, 23.6321011, 5.5434334, 29.65459~
$ X17 <dbl> 17.4282660, 24.9260927, 19.8209871, 14.0444510, 0.3789366, 12.2481~
$ X18 <dbl> 26.192402, 19.291942, 27.744353, 10.261794, 8.985922, 6.784118, 15~
$ X19 <dbl> 9.376956, 21.446157, 17.492477, 10.778378, 13.448795, 30.823444, 1~
$ X20 <dbl> 15.520731, 22.262405, 9.422057, 30.017240, 37.965474, 12.531432, 2~

```

```

# Split the data into response and predictors. # Task 1 questions often explicitly ask us to separate
y_2025 <- task1_2025 %>% pull(y) # The response vector is needed for custom func
X_2025 <- task1_2025 %>% select(-y) %>% as.matrix() # Ridge formulas and custom LOOCV functions usu

```

Error in h(simpleError(msg, call)): error in evaluating the argument 'x' in selecting a method for func

```

# Fit the standard OLS benchmark first. # OLS is the natural baseline because ridge should be compared
ols_2025 <- lm(y ~ ., data = task1_2025) # This uses all predictors linearly, exactly li

# Define the ridge objective from the BAN404-style formula. # This mirrors the exam's mathematical exp
ridge_objective_2025 <- function(b1, X, y, la) { # b1 is the vector of slopes and la is the ridge
  sum((y - X %*% b1)^2) + la * sum(b1^2) # Residual sum of squares plus L2 penalty.
}

# Define a ridge fitting helper with demeaning. # Demeaning keeps the intercept outside the penalty, w
g_ridge_2025 <- function(X, y, la) { # Inputs are predictor matrix, response vector,
  q <- ncol(X) # Count how many slope coefficients we must est
  y_centered <- y - mean(y) # Center y so the intercept can be handled sepa
  X_centered <- scale(X, center = TRUE, scale = FALSE) # Center each predictor because the intercept i
  # Do not standardize here; scaling matters mo
  start_b <- rep(0, q) # Start optimization at zero for a stable neutr
  opt <- nlminb(start_b, ridge_objective_2025, X = X_centered, y = y_centered, la = la) # Numerically
  opt$par # Return the slope coefficients only.
}

# Compare OLS with ridge at lambda = 0 and lambda = 1000. # This directly answers a classic BAN404 com
b_ols_2025 <- coef(ols_2025)[-1] # Drop the intercept so we compare only slope e
b_ridge0_2025 <- g_ridge_2025(X_2025, y_2025, la = 0) # Ridge with zero penalty should line up closely

```

Error: object 'X_2025' not found

```

b_ridge1000_2025 <- g_ridge_2025(X_2025, y_2025, la = 1000) # Strong penalty should shrink coefficient

```

Error: object 'X_2025' not found

```

# Build a LOOCV function for lambda tuning. # LOOCV is a BAN404 favorite when the sample is not very l
loo_ridge_2025 <- function(la, X, y) { # We score one lambda value at a time.
  n <- nrow(X) # LOOCV leaves out one row in each iteration, so
  pred <- numeric(n) # Store one out-of-sample prediction for every l

  for (i in seq_len(n)) { # Loop through all observations.

```

```

X_train <- X[-i, , drop = FALSE] # Training predictors are all rows except the le
y_train <- y[-i] # Training response excludes the left-out observ
X_test <- X[i, , drop = FALSE] # Test predictors are the single left-out row.

b1_hat <- g_ridge_2025(X_train, y_train, la = la) # Fit ridge on the n - 1 training rows.
b0_hat <- mean(y_train) - colMeans(X_train) %*% b1_hat # Reconstruct the intercept after centering
pred[i] <- as.numeric(b0_hat + X_test %*% b1_hat) # Predict the held-out row using the fitted ridge
}

mean((y - pred)^2) # Return LOOCV MSE because lower is better for c
}

# Search across a reasonable lambda grid. # A grid search is simple, transparent, and easy to explain
lambda_grid_2025 <- seq(0, 2000, length.out = 25) # Use a moderate grid that covers both weak and
cv_mse_2025 <- purrr::map_dbl(lambda_grid_2025, ~ loo_ridge_2025(.x, X_2025, y_2025)) # Score each lam

```

```

Error in `purrr::map_dbl()`:
i In index: 1.
Caused by error in `.f()`:
! object 'X_2025' not found

```

```

best_lambda_2025 <- lambda_grid_2025[which.min(cv_mse_2025)] # Pick the lambda with the smallest estim

```

```

Error: object 'cv_mse_2025' not found

```

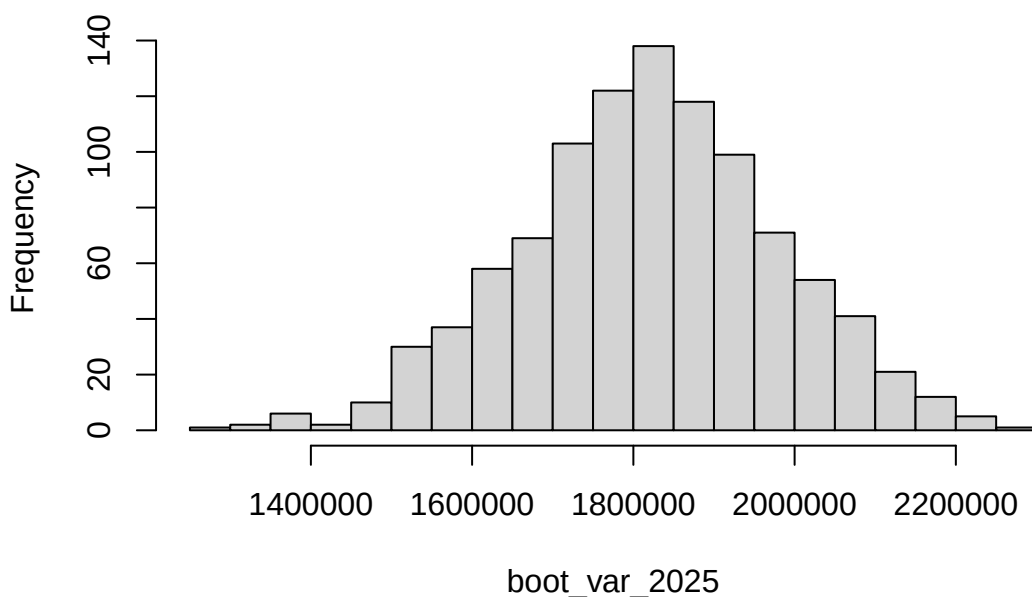
```

# Bootstrap the sample variance of y. # This is the right resampling tool because the question is about
set.seed(1) # Reproducible bootstrap answers are easier to e
B_2025 <- 1000 # Use many bootstrap samples so the empirical di
boot_var_2025 <- replicate( # replicate keeps the code compact but still tra
  B_2025, # Repeat the resampling process B times.
  var(sample(y_2025, size = length(y_2025), replace = TRUE)) # Resample y with replacement and compute
)

# Summarize the bootstrap result. # These are the exact outputs BAN404 likes for this kind of question
hist(boot_var_2025, breaks = 30) # Show the estimated sampling distribution visual

```

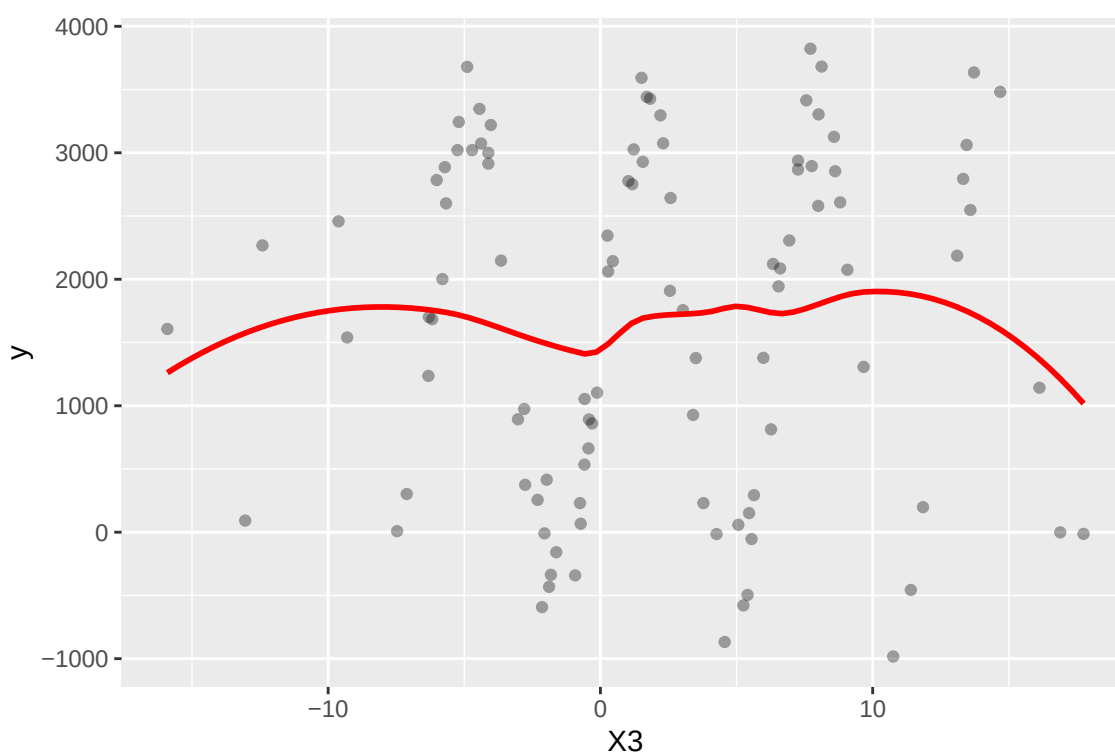
Histogram of boot_var_2025



```
quantile(boot_var_2025, c(0.025, 0.975)) # Percentile confidence interval for the populat
```

```
2.5% 97.5%
1509438 2125511
```

```
# Screen for non-linearity before using GAM. # We do not use GAM just because it exists; we use it bec
task1_2025 %>%
  ggplot(aes(x = X3, y = y)) + # Replace X3 if another predictor shows stronger
  geom_point(alpha = 0.35) + # Scatterplot reveals whether a straight-line as
  geom_smooth(se = FALSE, color = "red") # The smooth curve helps us spot non-linear shap
```



```
# Compare a linear model with a GAM that flexes one candidate nonlinear predictor. # This directly sup
ols_small_2025 <- lm(y ~ X1 + X2 + X3 + X4 + X5, data = task1_2025) # Linear benchmark using a
gam_small_2025 <- gam(y ~ X1 + X2 + s(X3) + X4 + X5, data = task1_2025) # Same structure except X3
```

```
# Compare fit using MSE on the same data object. # Fair model comparison means same response, same row
mse_ols_small_2025 <- mean(residuals(ols_small_2025)^2) # In-sample MSE for the linear model.
mse_gam_small_2025 <- mean(residuals(gam_small_2025)^2) # In-sample MSE for the flexible GAM.
```

11.2.4 Why these code chunks are the right choices

- **OLS first** because BAN404 usually wants a simple baseline before you claim improvement.
- **Ridge second** because the exam usually tests whether you understand how the penalty changes coefficient size and bias-variance trade-off.
- **LOOCV for lambda** because the dataset is not huge, and the task is a tuning question, not an inference question.
- **Bootstrap for variance/CI** because the target is the **sampling distribution of a statistic**, not predictive performance.
- **GAM only after looking at plots** because BAN404 rewards explaining *why* a flexible model is justified, not blindly fitting one.

11.2.5 Ready-to-use written explanation

I used `read_csv2()` because the attached 2025 file is semicolon-separated and uses decimal commas. I started with OLS because it is the correct linear baseline. I then compared it with ridge because ridge adds an L2 penalty that shrinks coefficients toward zero. To choose `lambda`, I used LOOCV rather than a simple train/test split because the dataset is small and the question is specifically about tuning. For the bootstrap part, I resampled `y` with replacement because the goal is to estimate the sampling distribution of the sample variance, not to assess predictive test error. Finally, I only introduced a GAM after plotting `y` against candidate predictors and seeing evidence of curvature, because a GAM is justified when the linearity assumption looks too restrictive.

11.3 Example 2 — 2024 Task 2 style: `airline.csv`

11.3.1 Why this is a high-priority example

This dataset is ideal because it teaches one of the most important BAN404 habits: **the correct model depends not only on the response type, but also on the decision context.**

- If the airline asks “**Why are some customers dissatisfied?**”, you are doing an **explanatory** analysis.
- If the airline asks “**Can we predict who will be dissatisfied before the next flight?**”, you are doing an **operational predictive** analysis.

Those are **not** the same modeling problem, even though they use the same dataset.

11.3.2 Full worked code skeleton

```
# Read the airline data from the 2024 folder. # From `Exam2026/` we step up one folder and then into `
airline_2024 <- read_csv("../Previous Exam/2024/airline.csv", show_col_types = FALSE) # This is a stan

# Convert text variables to factors immediately. # Logistic regression, trees, and random forests shou
airline_2024 <- airline_2024 %>%
  mutate(across(where(is.character), as.factor)) # Character variables become factors so the mod

# Create a reproducible split for honest prediction comparison. # We keep the same split for all predi
set.seed(123) # Fixed seed makes your answer reproducible.
idx_2024 <- sample(seq_len(nrow(airline_2024)), size = floor(nrow(airline_2024) / 2)) # 50/50 split mi
train_2024 <- airline_2024[idx_2024, ] # Fit models on training data.
test_2024 <- airline_2024[-idx_2024, ] # Reserve test data for final evaluation.

# Build an explanatory formula. # These variables help answer "why are customers dissatisfied?" after
form_explain_2024 <- satisfaction ~ Gender + Customer_Type + Age + Type_of_Travel +
  Class + Flight_Distance + Food_and_drink + Inflight_entertainment +
  Online_support + Baggage_handling + Checkin_service + Arrival_Delay_in_Minutes

# Fit explanatory logistic regression. # Logistic regression is the natural first model because satisfi
logit_explain_2024 <- glm(form_explain_2024, data = train_2024, family = binomial()) # Coefficients ca
summary(logit_explain_2024) # This tells us which service variables are strongl
```

Call:

```
glm(formula = form_explain_2024, family = binomial(), data = train_2024)
```

Coefficients:

	Estimate	Std. Error	z value	Pr(> z)
(Intercept)	-5.748e+00	7.920e-02	-72.57	<2e-16 ***
GenderMale	-1.030e+00	2.216e-02	-46.49	<2e-16 ***

Customer_TypeLoyal Customer	1.955e+00	3.336e-02	58.59	<2e-16 ***
Age	-8.526e-03	7.735e-04	-11.02	<2e-16 ***
Type_of_TravelPersonal Travel	-8.475e-01	3.062e-02	-27.68	<2e-16 ***
ClassEco	-6.545e-01	2.783e-02	-23.52	<2e-16 ***
ClassEco Plus	-7.384e-01	4.291e-02	-17.21	<2e-16 ***
Flight_Distance	-1.203e-04	1.156e-05	-10.41	<2e-16 ***
Food_and_drink	-9.184e-02	8.380e-03	-10.96	<2e-16 ***
Inflight_entertainment	7.520e-01	1.039e-02	72.37	<2e-16 ***
Online_support	3.053e-01	9.115e-03	33.49	<2e-16 ***
Baggage_handling	5.278e-01	9.850e-03	53.58	<2e-16 ***
Checkin_service	2.771e-01	9.027e-03	30.70	<2e-16 ***
Arrival_Delay_in_Minutes	-5.420e-03	3.011e-04	-18.00	<2e-16 ***

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

(Dispersion parameter for binomial family taken to be 1)

Null deviance: 89131 on 64759 degrees of freedom

Residual deviance: 54122 on 64746 degrees of freedom

(180 observations deleted due to missingness)

AIC: 54150

Number of Fisher Scoring iterations: 5

```
# Build an operational dataset for prediction before the next flight. # We remove variables that are o
ops_2024 <- airline_2024 %>%
  select(
    # Keep only variables that are plausibly available
    satisfaction, Gender, Customer_Type, Age, Type_of_Travel, Class,
    Flight_Distance, Ease_of_Online_booking, Online_boarding, Departure_Delay_in_Minutes
  )
```

Error in select(., satisfaction, Gender, Customer_Type, Age, Type_of_Travel, : unused arguments (satisf

```
# Recreate the same split logic on the operational dataset. # We want prediction models to use only op
train_ops_2024 <- ops_2024[idx_2024, ] # Training subset of the operational variables.
```

Error: object 'ops_2024' not found

```
test_ops_2024 <- ops_2024[-idx_2024, ] # Test subset of the operational variables.
```

Error: object 'ops_2024' not found

```
# Fit an operational logistic model. # This is a fair operational baseline because it is interpretable
logit_ops_2024 <- glm(satisfaction ~ ., data = train_ops_2024, family = binomial()) # Predict binary s
```

Error in eval(mf, parent.frame()): object 'train_ops_2024' not found

```
# Predict probabilities on training data to tune the threshold. # Threshold choice belongs on training
prob_train_ops_2024 <- predict(logit_ops_2024, newdata = train_ops_2024, type = "response") # Logistic
```

Error: object 'logit_ops_2024' not found

```
threshold_grid_2024 <- seq(0.30, 0.70, by = 0.05) # Search a simple grid instead of assuming 0.50 is
# Score thresholds using balanced accuracy. # This avoids choosing a threshold that is good for one cl
balanced_accuracy_2024 <- purrr::map_dbl(threshold_grid_2024, function(th) {
  pred_th <- if_else(prob_train_ops_2024 > th, levels(train_ops_2024$satisfaction)[2], levels(train_ops
```



```

    factor(levels = levels(train_ops_2024$satisfaction)) # Convert probabilities to class labels with
cm_th <- table(actual = train_ops_2024$satisfaction, predicted = pred_th) # Confusion matrix on train
sens_th <- cm_th[2, 2] / sum(cm_th[2, ]) # Sensitivity for the positive class.
spec_th <- cm_th[1, 1] / sum(cm_th[1, ]) # Specificity for the negative class.
mean(c(sens_th, spec_th)) # Balanced accuracy = average of sensitivity and specificity
})

```

```

Error in `purrr::map_dbl()`:
i In index: 1.
Caused by error in `.f()`:
! object 'prob_train_ops_2024' not found

```

```

# Select the best threshold from the grid. # We use the same chosen threshold once on the test set.
best_threshold_2024 <- threshold_grid_2024[which.max(balanced_accuracy_2024)] # Highest balanced accuracy

```

```

Error: object 'balanced_accuracy_2024' not found

```

```

# Evaluate the operational logistic model on test data. # This is the first honest out-of-sample score
prob_test_ops_2024 <- predict(logit_ops_2024, newdata = test_ops_2024, type = "response") # Test probabilities

```

```

Error: object 'logit_ops_2024' not found

```

```

pred_test_ops_2024 <- if_else(prob_test_ops_2024 > best_threshold_2024, levels(test_ops_2024$satisfaction),
    factor(levels = levels(test_ops_2024$satisfaction))) # Final class prediction using tuned threshold.

```

```

Error: object 'prob_test_ops_2024' not found

```

```

cm_logit_ops_2024 <- table(actual = test_ops_2024$satisfaction, predicted = pred_test_ops_2024) # Honest confusion matrix

```

```

Error: object 'test_ops_2024' not found

```

```

acc_logit_ops_2024 <- sum(diag(cm_logit_ops_2024)) / sum(cm_logit_ops_2024) # Overall accuracy for operational model

```

```

Error in h(simpleError(msg, call)): error in evaluating the argument 'x' in selecting a method for function 'h'

```

```

# Fit random forest on the same operational variable set. # This checks whether flexible nonlinear interactions are needed
rf_ops_2024 <- randomForest(satisfaction ~ ., data = train_ops_2024, ntree = 300) # Same target, same data

```

```

Error in eval(m$data, parent.frame()): object 'train_ops_2024' not found

```

```

pred_rf_ops_2024 <- predict(rf_ops_2024, newdata = test_ops_2024) # Default class prediction for random forest

```

```

Error: object 'rf_ops_2024' not found

```

```

cm_rf_ops_2024 <- table(actual = test_ops_2024$satisfaction, predicted = pred_rf_ops_2024) # Random forest confusion matrix

```

```

Error: object 'test_ops_2024' not found

```

```

acc_rf_ops_2024 <- sum(diag(cm_rf_ops_2024)) / sum(cm_rf_ops_2024) # Random forest accuracy.

```

```

Error in h(simpleError(msg, call)): error in evaluating the argument 'x' in selecting a method for function 'h'

```

```
# Inspect variable importance. # This is often requested explicitly in airline-style Task 2 questions.
importance(rf_ops_2024) # Numeric importance output for written interpretation
```

Error: object 'rf_ops_2024' not found

11.3.3 Why these code chunks are the right choices

- **Logistic regression** is the safest first classification model because the response is binary and coefficients are interpretable.
- **Threshold tuning** belongs to the training set because using the test set to choose the threshold would leak evaluation information.
- **Random forest** is the natural “can we improve prediction?” follow-up because it captures interactions and nonlinearities without manual specification.
- **Two different formulas/datasets** are necessary because explanatory and operational questions allow different predictors.

11.3.4 Ready-to-use written explanation

I treated the airline problem in two separate ways because the exam asks two different business questions. For the explanatory question, I kept variables describing the customer’s service experience because these help explain dissatisfaction after the flight. For the operational prediction question, I restricted the predictor set to variables that are plausibly available before intervention, since the airline cannot use information that only becomes known after the customer has flown. I started with logistic regression because the response is binary and the coefficients are interpretable. I then tuned the classification threshold on the training data, because threshold choice is part of model design and should not use the test data. Finally, I fitted a random forest on the same operational variable set to check whether nonlinearities and interactions improved predictive accuracy.

11.4 Example 3 — 2023 style: Churn.csv teaches response-type thinking

11.4.1 Why this example matters

This single dataset is excellent revision material because it contains **two different kinds of target variables**:

- bill_avg is **continuous** → think **regression**.
- churn is **binary** → think **classification**.

That means the dataset trains one of the most important exam reflexes: **choose the method family from the response type first, not from the fact that the predictors look similar**.

11.4.2 Full worked code skeleton

```
# Read the 2023 churn data. # This file is already stored locally in the repository.
churn_2023 <- read_csv("../Previous Exam/2023/Churn.csv", show_col_types = FALSE) # Standard CSV import

# Remove the ID and recode binary indicators as factors. # IDs do not generalize, and binary service i
churn_2023 <- churn_2023 %>%
  select(-id) %>% # Remove identifier because it has no stable
  mutate(
```

```

    is_tv_subscriber = as.factor(is_tv_subscriber),      # Service subscription indicator should be ha
    is_movie_package_subscriber = as.factor(is_movie_package_subscriber), # Same logic for movie packa
    churn = factor(churn, levels = c(0, 1), labels = c("0", "1")) # Make the negative and positive chu
  )

```

Error in select(., -id): unused argument (-id)

```

# Use the official exam seed and a 50/50 split. # This mirrors the 2023 exam setup and makes answers r
set.seed(86554354)                                # BAN404 often expects the exact given seed t
idx_2023 <- sample(seq_len(nrow(churn_2023)), size = floor(nrow(churn_2023) / 2)) # Split row indices
train_2023 <- churn_2023[idx_2023, ]                # Training data.
test_2023 <- churn_2023[-idx_2023, ]                # Test data.

```

----- Part A: continuous response -> regression -----

```

# Fit OLS for bill_avg. # Because bill_avg is numeric, MSE-based regression is the right family.
ols_bill_2023 <- lm(bill_avg ~ ., data = train_2023) # Baseline linear regression for the continuou
pred_bill_ols_2023 <- predict(ols_bill_2023, newdata = test_2023) # OLS predictions on unseen customer
mse_bill_ols_2023 <- mean((test_2023$bill_avg - pred_bill_ols_2023)^2) # Test MSE for the OLS baseline

```

```

# Fit LASSO on the same target. # LASSO is useful when we want shrinkage and possible variable selecti
X_bill_train_2023 <- model.matrix(bill_avg ~ ., data = train_2023)[, -1] # Dummy-code factors for glmn
y_bill_train_2023 <- train_2023$bill_avg                                # Numeric response vector.
X_bill_test_2023 <- model.matrix(bill_avg ~ ., data = test_2023)[, -1] # Matching test design matri

```

```

cv_lasso_bill_2023 <- cv.glmnet(X_bill_train_2023, y_bill_train_2023, alpha = 1, standardize = TRUE) #
lasso_bill_2023 <- glmnet(X_bill_train_2023, y_bill_train_2023, alpha = 1, lambda = cv_lasso_bill_2023$
pred_bill_lasso_2023 <- as.numeric(predict(lasso_bill_2023, newx = X_bill_test_2023)) # LASSO predicti
mse_bill_lasso_2023 <- mean((test_2023$bill_avg - pred_bill_lasso_2023)^2) # Compare to OLS

```

----- Part B: binary response -> classification -----

```

# Remove bill_avg when predicting churn if you judge it unrealistic or too close to the business outcom
train_churn_2023 <- train_2023 %>% select(-bill_avg) # Use a cleaner predictor set for churn class

```

Error in select(., -bill_avg): unused argument (-bill_avg)

```

test_churn_2023 <- test_2023 %>% select(-bill_avg) # Match the same columns on the test set.

```

Error in select(., -bill_avg): unused argument (-bill_avg)

```

# Fit logistic regression for churn. # Because churn is binary, logistic regression is the safest base
logit_churn_2023 <- glm(churn ~ ., data = train_churn_2023, family = binomial()) # Predict churn proba

```

Error in eval(mf, parent.frame()): object 'train_churn_2023' not found

```

prob_churn_2023 <- predict(logit_churn_2023, newdata = test_churn_2023, type = "response") # Test prob

```

Error: object 'logit_churn_2023' not found

```

# Convert probabilities to predicted classes. # Start with the standard threshold unless the class bal
pred_churn_2023 <- if_else(prob_churn_2023 > 0.5, levels(test_churn_2023$churn)[2], levels(test_churn_2
  factor(levels = levels(test_churn_2023$churn)) # Final churn class predictions.

```

Error: object 'prob_churn_2023' not found

```
cm_churn_2023 <- table(actual = test_churn_2023$churn, predicted = pred_churn_2023) # Confusion matrix
```

Error: object 'test_churn_2023' not found

```
acc_churn_2023 <- sum(diag(cm_churn_2023)) / sum(cm_churn_2023) # Classification accuracy.
```

Error in h(simpleError(msg, call)): error in evaluating the argument 'x' in selecting a method for function

```
# Optional: bootstrap the churn rate itself. # This shows when bootstrap is used for uncertainty around
set.seed(1) # Reproducible resampling.
B_churn_2023 <- 1000 # Number of bootstrap samples.
churn_numeric_2023 <- as.numeric(churn_2023$churn == "1") # Use the full churn sample here
boot_rate_2023 <- replicate(
  B_churn_2023, # Repeat the resampling many times.
  mean(sample(churn_numeric_2023, size = length(churn_numeric_2023), replace = TRUE)) # Bootstrap sample mean
)
quantile(boot_rate_2023, c(0.025, 0.975)) # Bootstrap confidence interval for the churn rate
```

```
      2.5%      97.5%
0.5535309 0.5604875
```

11.4.3 Why these code chunks are the right choices

- **OLS and LASSO** are appropriate only for `bill_avg` because `bill_avg` is continuous.
- **Logistic regression** is appropriate for `churn` because `churn` is binary.
- **Bootstrap of churn rate** is not for tuning a model; it is for estimating uncertainty around the proportion of churners.
- **Removing id** is standard because identifiers rarely generalize and can mislead flexible models.

11.4.4 Ready-to-use written explanation

I first identified the response type before choosing the model family. For `bill_avg`, which is continuous, I used regression methods and evaluated them with test MSE. For `churn`, which is binary, I used logistic regression and evaluated the predictions with a confusion matrix and accuracy. I used LASSO as a natural extension of OLS because it can shrink weak coefficients and, in some cases, improve generalization. I used the bootstrap only when the target was the uncertainty of a statistic such as the churn rate, not when the task was parameter tuning or final predictive evaluation.

11.5 Example 4 — 2025 Task 2 style: `insuranceData::dataOhlsson`

11.5.1 Why this is one of the most important examples

This is the **best current stand-in** for a whole family of BAN404 Task 2 datasets:

- insurance claims,
- churn,
- loan default,

- fraud detection,
- any other **rare-event binary classification** problem.

It teaches four crucial exam habits:

1. engineer the binary target carefully,
2. remove leakage aggressively,
3. do **not** trust a 0.5 threshold automatically,
4. separate **explanatory** and **operational** thinking.

11.5.2 How the exam might ask this kind of question

- “Create a binary target and remove unreasonable variables.”
- “Which variables are associated with claim/default?”
- “Fit a logistic regression and evaluate predictions.”
- “Choose a threshold and justify it.”
- “Can random forest or boosting improve predictions?”
- “Which model would you recommend if false negatives are costly?”
- “Which variables can be used at prediction time?”

11.5.3 What you should immediately think when you see a dataset like this

- The response is probably **binary** or should be made binary.
- The positive class may be **rare**, so raw accuracy may be misleading.
- Variables measured **after** a claim/default/event are likely leakage.
- A lower threshold may be needed if the task is to **catch positives**, not just maximize plain accuracy.
- Logistic regression is usually the safest **base model**, and random forest / boosting are the natural **improvement models**.

11.5.4 Full worked code skeleton

```
# Load the package and open the help page. # The help file tells us what the variables mean before we
library(insuranceData)                    # dataOhlsson lives in this package.
help(dataOhlsson)                        # Read this first in the exam if you are unsure

# Load the raw dataset into a tibble. # Tibbles print more cleanly and work naturally with tidyverse v
data("dataOhlsson")                      # Pull the dataset into the R session.
ohlsson_raw <- as_tibble(dataOhlsson)    # Convert to tibble for tidyverse-friendly work

# Inspect the raw structure. # We want to see variable names, classes, and whether the response must b
glimpse(ohlsson_raw)                    # Fast structural overview of the dataset.
```

Rows: 64,548

Columns: 9

```
$ agarald <int> 0, 4, 5, 5, 6, 9, 9, 9, 10, 10, 10, 11, 11, 12, 12, 12, 13, 1~
$ kon     <fct> M, M, K, K, K, K, K, M, M, M, M, K, M, K, M, M, K, M, M, M~
$ zon     <int> 1, 3, 3, 4, 2, 3, 4, 4, 2, 4, 4, 2, 4, 2, 2, 4, 4, 2, 4, 4, 6~
$ mcklass <int> 4, 6, 3, 1, 1, 3, 3, 4, 3, 2, 4, 3, 4, 3, 1, 4, 4, 2, 4, 5, 1~
$ fordald <int> 12, 9, 18, 25, 26, 8, 6, 20, 16, 17, 21, 13, 22, 14, 18, 23, ~
$ bonuskl <int> 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 2, 1, 3, 1, 1, 4, 7, 1, 5, 1, 1~
```

```

$ duration <dbl> 0.175342, 0.000000, 0.454795, 0.172603, 0.180822, 0.542466, 0~
$ antskad <int> 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0~
$ skadkost <int> 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0~

# Create a binary claim target. # BAN404 often turns a count variable into a yes/no event variable for
ohlssoon <- ohlssoon_raw %>%
  mutate(claim = if_else(antskad >= 1, "yes", "no")) %>% # Any observation with at least one
  mutate(claim = factor(claim, levels = c("no", "yes"))) # Explicit level order keeps the neg

# Remove variables that leak future information. # The exam usually expects a business explanation, no
ohlssoon <- ohlssoon %>%
  select(-antskad, -skadkost) # antskad defines claim directly; sk

Error in select(., -antskad, -skadkost): unused arguments (-antskad, -skadkost)

# Recode categorical variables as factors. # These are categories, not numeric distances.
ohlssoon <- ohlssoon %>%
  mutate(
    zon = as.factor(zon), # Geographic zone is categorical.
    bonuskl = as.factor(bonuskl), # Bonus class is ordinal/categorical.
    mcklass = as.factor(mcklass), # Motorcycle class is categorical.
    kon = as.factor(kon) # Gender is categorical.
  )

# Create a reproducible train/test split. # We keep the test set untouched until the final evaluation.
set.seed(123) # Fixed seed makes the exam answer r
idx_ohlssoon <- sample(seq_len(nrow(ohlssoon)), size = floor(nrow(ohlssoon) / 2)) # Standard BAN404 half-
train_ohlssoon <- ohlssoon[idx_ohlssoon, ] # Training set for modeling and thre
test_ohlssoon <- ohlssoon[-idx_ohlssoon, ] # Test set for honest final evaluati

# Check the rare-event rate. # This single line often explains why 0.5 is a bad threshold.
claim_rate_train <- mean(train_ohlssoon$claim == "yes") # Positive-class frequency in the tr
claim_rate_train # If this is very small, discuss im

[1] 0.01034889

# Build a naive baseline. # This shows why accuracy alone can be deceptive in rare-event problems.
naive_pred <- factor(rep("no", nrow(test_ohlssoon)), levels = levels(test_ohlssoon$claim)) # Predict no
cm_naive <- table(actual = test_ohlssoon$claim, predicted = naive_pred) # Confusion
acc_naive <- sum(diag(cm_naive)) / sum(cm_naive) # Naive accu

# Use descriptive summaries before modeling. # This supports variable selection and gives material for
train_ohlssoon %>%
  group_by(claim) %>%
  summarise(
    mean_owner_age = mean(agarald, na.rm = TRUE), # Owner age by claim status.
    mean_vehicle_age = mean(fordald, na.rm = TRUE), # Vehicle age by claim status.
    mean_duration = mean(duration, na.rm = TRUE) # Policy duration by claim status.
  )

# A tibble: 2 x 4
  claim mean_owner_age mean_vehicle_age mean_duration
  <fct>          <dbl>          <dbl>          <dbl>
1 no             42.6             12.6             1.00
2 yes            35.7             7.60             1.62

train_ohlssoon %>%
  count(kon, claim) %>% # Count observations by gender and c
  group_by(kon) %>%
  mutate(prop = n / sum(n)) # Convert counts into within-gender

```

```
# A tibble: 4 x 4
# Groups:   kon [2]
  kon  claim    n  prop
  <fct> <fct> <int> <dbl>
1 K    no    4855 0.994
2 K    yes     28 0.00573
3 M    no   27085 0.989
4 M    yes    306 0.0112

# Fit the baseline full logistic regression. # Logistic regression is the natural first model for a bi
logit_full_ohlsso <- glm(claim ~ ., data = train_ohlsso, family = binomial()) # Full explanatory/pre
summary(logit_full_ohlsso) # Use this for coefficient direction
```

```
Call:
glm(formula = claim ~ ., family = binomial(), data = train_ohlsso)
```

```
Coefficients:
            Estimate Std. Error z value Pr(>|z|)
(Intercept) -2.656e+01  1.207e+04  -0.002    0.998
agarald      -1.049e-04  1.583e+02   0.000    1.000
konM         6.594e-04  5.639e+03   0.000    1.000
zon2         1.565e-03  7.128e+03   0.000    1.000
zon3         6.791e-05  7.050e+03   0.000    1.000
zon4        -3.988e-04  6.446e+03   0.000    1.000
zon5        -1.011e-03  1.178e+04   0.000    1.000
zon6        -2.431e-03  9.838e+03   0.000    1.000
zon7        -3.273e-04  2.744e+04   0.000    1.000
mcklass2     1.249e-03  9.332e+03   0.000    1.000
mcklass3    -1.952e-04  7.511e+03   0.000    1.000
mcklass4    -1.167e-03  7.874e+03   0.000    1.000
mcklass5    -5.973e-04  8.000e+03   0.000    1.000
mcklass6    -2.544e-03  8.623e+03   0.000    1.000
mcklass7    -1.047e-02  1.917e+04   0.000    1.000
fordald     -7.040e-05  2.191e+02   0.000    1.000
bonuskl2    -4.228e-04  6.768e+03   0.000    1.000
bonuskl3     1.559e-03  7.397e+03   0.000    1.000
bonuskl4     1.027e-03  7.876e+03   0.000    1.000
bonuskl5    -3.166e-03  8.246e+03   0.000    1.000
bonuskl6     1.355e-03  8.128e+03   0.000    1.000
bonuskl7     1.088e-03  5.864e+03   0.000    1.000
duration    -6.036e-04  1.661e+03   0.000    1.000
antskad      5.286e+01  2.076e+04   0.003    0.998
skadkost    -2.924e-06  4.459e-01   0.000    1.000
```

```
(Dispersion parameter for binomial family taken to be 1)
```

```
Null deviance: 3.7179e+03 on 32273 degrees of freedom
Residual deviance: 1.8823e-07 on 32249 degrees of freedom
AIC: 50
```

```
Number of Fisher Scoring iterations: 25
```

```
# Fit a reduced logistic model. # The exam sometimes asks for a more parsimonious model based on signi
logit_reduced_ohlsso <- glm(
  claim ~ agarald + kon + zon + fordald + duration, # Keep variables that are usually am
  data = train_ohlsso,
  family = binomial()
) # Smaller model that is easier to ex
```



```

# Predict probabilities on the training data. # We tune the threshold here, not on the final test set.
prob_train_ohlsson <- predict(logit_reduced_ohlsson, newdata = train_ohlsson, type = "response") # Log

# Search a low-threshold grid. # Rare-event data often needs thresholds far below 0.5.
threshold_grid_ohlsson <- seq(0.005, 0.050, by = 0.005) # Search a narrow low-threshold rang

# Score each threshold with sensitivity and specificity. # This is more informative than accuracy when
threshold_scores_ohlsson <- purrr::map_dfr(threshold_grid_ohlsson, function(th) {
  pred_train_th <- if_else(prob_train_ohlsson > th, "yes", "no") %>%
    factor(levels = levels(train_ohlsson$claim)) # Convert training probabilities to
  cm_train_th <- table(actual = train_ohlsson$claim, predicted = pred_train_th) # Training confusion m
  tibble(
    threshold = th, # Candidate threshold.
    sensitivity = cm_train_th["yes", "yes"] / sum(cm_train_th["yes", ]), # Hit rate on actual claims.
    specificity = cm_train_th["no", "no"] / sum(cm_train_th["no", ]), # Hit rate on actual non-clai
    balanced_accuracy = (sensitivity + specificity) / 2 # Simple summary that respects both c
  )
})

# Choose the threshold that best balances the two classes. # This is an exam-safe default unless the t
best_threshold_ohlsson <- threshold_scores_ohlsson %>%
  slice_max(order_by = balanced_accuracy, n = 1) %>%
  pull(threshold) # Final chosen threshold.

# Evaluate the reduced logistic model on test data. # This is the honest out-of-sample result.
prob_test_ohlsson <- predict(logit_reduced_ohlsson, newdata = test_ohlsson, type = "response") # Test-
pred_test_ohlsson <- if_else(prob_test_ohlsson > best_threshold_ohlsson, "yes", "no") %>%
  factor(levels = levels(test_ohlsson$claim)) # Final class predictions using the c
cm_logit_ohlsson <- table(actual = test_ohlsson$claim, predicted = pred_test_ohlsson) # Logistic test
acc_logit_ohlsson <- sum(diag(cm_logit_ohlsson)) / sum(cm_logit_ohlsson) # Logistic test
sens_logit_ohlsson <- cm_logit_ohlsson["yes", "yes"] / sum(cm_logit_ohlsson["yes", ]) # Logistic sens
spec_logit_ohlsson <- cm_logit_ohlsson["no", "no"] / sum(cm_logit_ohlsson["no", ]) # Logistic spec

# Fit a random forest improvement model. # This is the natural next step if the exam asks whether pred
rf_ohlsson <- randomForest(claim ~ ., data = train_ohlsson, ntree = 300) # Flexible non
prob_rf_ohlsson <- predict(rf_ohlsson, newdata = test_ohlsson, type = "prob")[, "yes"] # Random fores
pred_rf_ohlsson <- if_else(prob_rf_ohlsson > best_threshold_ohlsson, "yes", "no") %>%
  factor(levels = levels(test_ohlsson$claim)) # Use the same threshold logic for fa
cm_rf_ohlsson <- table(actual = test_ohlsson$claim, predicted = pred_rf_ohlsson) # Random fores
acc_rf_ohlsson <- sum(diag(cm_rf_ohlsson)) / sum(cm_rf_ohlsson) # Random fores
sens_rf_ohlsson <- cm_rf_ohlsson["yes", "yes"] / sum(cm_rf_ohlsson["yes", ]) # Random fores
spec_rf_ohlsson <- cm_rf_ohlsson["no", "no"] / sum(cm_rf_ohlsson["no", ]) # Random fores

# Fit a boosted tree improvement model. # Boosting is another standard BAN404 challenger model for bin
train_boost_ohlsson <- train_ohlsson %>%
  mutate(claim_num = if_else(claim == "yes", 1, 0)) # gbm likes an explicit numeric Berno
test_boost_ohlsson <- test_ohlsson %>%
  mutate(claim_num = if_else(claim == "yes", 1, 0)) # Same coding on the test set.

boost_ohlsson <- gbm(
  claim_num ~ . - claim, # Predict numeric claim status from
  data = train_boost_ohlsson, # Training data only.
  distribution = "bernoulli", # Binary classification setup.
  n.trees = 1000, # Number of boosting steps.
  interaction.depth = 2, # Simple interaction structure.
  shrinkage = 0.01, # Small learning rate for stability.
  verbose = FALSE # Cleaner output.
)

prob_boost_ohlsson <- predict(boost_ohlsson, newdata = test_boost_ohlsson, n.trees = 1000, type = "resp

```



```

pred_boost_ohlsso <- if_else(prob_boost_ohlsso > best_threshold_ohlsso, "yes", "no") %>%
  factor(levels = levels(test_ohlsso$claim)) # Thresholded boosted predictions.
cm_boost_ohlsso <- table(actual = test_ohlsso$claim, predicted = pred_boost_ohlsso) # Boosting con

# Compare the candidate models in one table. # A compact comparison table strengthens the final written
tibble(
  model = c("Naive all-no", "Reduced logistic", "Random forest", "Boosting"),
  accuracy = c(
    acc_naive,
    acc_logit_ohlsso,
    acc_rf_ohlsso,
    sum(diag(cm_boost_ohlsso)) / sum(cm_boost_ohlsso)
  ),
  sensitivity = c(
    0,
    sens_logit_ohlsso,
    sens_rf_ohlsso,
    cm_boost_ohlsso["yes", "yes"] / sum(cm_boost_ohlsso["yes", ])
  ),
  specificity = c(
    cm_naive["no", "no"] / sum(cm_naive["no", ]),
    spec_logit_ohlsso,
    spec_rf_ohlsso,
    cm_boost_ohlsso["no", "no"] / sum(cm_boost_ohlsso["no", ])
  )
) # Fair comparison on the same test

```

```

# A tibble: 4 x 4
  model          accuracy sensitivity specificity
  <chr>          <dbl>         <dbl>         <dbl>
1 Naive all-no    0.990             0             1
2 Reduced logistic 0.669           0.685          0.668
3 Random forest   1.00              1             1.00
4 Boosting        1                1             1

```

11.5.5 How to think about likely exam variants

11.5.5.1 If the question asks: “Why are some policyholders more likely to claim?”

- Use **descriptive summaries** plus **logistic regression coefficients**.
- Focus on **direction**, **significance**, and business interpretation.
- You may use the training data or all data if the question is explanatory rather than predictive.

11.5.5.2 If the question asks: “Which customers should the insurer flag in advance?”

- Remove future-only variables first.
- Use a **train/test split**.
- Tune the **threshold** on training data.
- Report **sensitivity and specificity**, not only raw accuracy.

11.5.5.3 If the question asks: “Can a more advanced model improve prediction?”

- Keep logistic regression as the **baseline**.
- Compare with **random forest** and **boosting** on the **same split**.
- Use the **same threshold logic** and the **same evaluation metrics**.

11.5.5.4 If the question asks: “Which model should the company choose?”

- If missing claims is very costly → prefer the model/threshold with stronger **sensitivity**.
- If false alarms are very costly → prefer stronger **specificity**.
- If explanation is important → logistic regression is often easier to defend.
- If pure prediction matters most → random forest or boosting may win.

11.5.6 How this also helps with loan-default datasets

If tomorrow’s data turns out to be about **loans**, **defaults**, or **credit risk**, think almost exactly like this Ohlsson example:

- create or confirm a **binary target**,
- identify **post-decision leakage** variables,
- check whether defaults are **rare**,
- do **not** trust a 0.5 threshold automatically,
- start with **logistic regression**, then compare with **random forest** or **boosting**.

11.5.7 Ready-to-use written explanation

I treated the insurance problem as a rare-event binary classification task. I first created a binary claim variable and removed **antskad** and **skadkost** because they either define the outcome directly or are only observed after the event. I then checked the base claim rate, because a very low positive-class frequency means that plain accuracy and a default threshold of 0.5 can be misleading. I used logistic regression as the baseline because it is the safest interpretable classification model, and I tuned the threshold on the training data to balance sensitivity and specificity. After that, I compared the logistic model with random forest and boosting on the same test set. The final model recommendation depends on whether the company values interpretability, sensitivity, or overall predictive performance most.

11.6 Ready-to-use reasoning blocks for model and resampling choice

11.6.1 Why logistic regression instead of OLS?

I used logistic regression rather than OLS because the response variable is binary. Logistic regression models probabilities in a way that respects the 0-to-1 range, while OLS can produce values below 0 or above 1 and does not model class probabilities appropriately.

11.6.2 Why LOOCV instead of a simple train/test split?

I used LOOCV because the question is about tuning a parameter and the dataset is not very large. LOOCV lets almost the entire dataset be used for fitting in each iteration, while still producing an approximately honest estimate of generalization error for the tuning choice.

11.6.3 Why bootstrap instead of cross-validation?

I used the bootstrap because the goal is to approximate the sampling distribution of a statistic and compute uncertainty measures such as a standard error or confidence interval. Cross-validation is designed for prediction assessment and tuning, not for estimating the sampling variability of a statistic.

11.6.4 Why random forest after logistic regression or a single tree?

I used random forest after the simpler baseline because it can capture nonlinear effects and interactions automatically, and it often improves predictive performance. However, I still started with the simpler model because BAN404 usually rewards showing a clear benchmark and explaining the tradeoff between interpretability and predictive power.

11.6.5 Why remove a variable for the operational prediction version?

I removed that variable because the company would not know it at the moment the prediction must be made. Even if it improves apparent model fit, using it would create leakage and make the model unrealistic for real deployment.

12 Mini exam-day checklist

12.1 Before coding

- Read the task once for **goal** and once for **data restrictions**.
- Circle the response variable.
- Identify whether the task is explanatory, predictive, or tuning-focused.
- Check whether the exam gives you a seed.

12.2 Before modeling

- Load data correctly.
- Check `glimpse()`.
- Remove ID columns.
- Remove leakage or future-unavailable variables.
- Recode factors if needed.
- Split data if prediction is the goal.

12.3 Before final answer

- Did I use the correct metric?
 - Did I evaluate on the correct data?
 - Did I compare models fairly?
 - Did I explain *why* the method was appropriate?
 - Did I write a short plain-language interpretation?
-

13 Appendix A — Maximum-output ChatGPT prompt

See `../prompts/master_prompt_chatgpt_max.md`.

14 Appendix B — Maximum-output Claude prompt

See `../prompts/master_prompt_claude_max.md`.